Universidad de Sevilla

Programa de Doctorado en Bioinformática

Departamento de Biología Computacional e Informática Científica

PROTEA: A Framework for Scalable Protein Functional Annotation

Doctoral Thesis

submitted in partial fulfilment
of the requirements for the degree of
Doctor of Philosophy

Author
Francisco Miguel Pérez Canales

Co-supervisor
David Orellana-Martín

Co-supervisor
Ana M. Rojas

Academic Year 2025/2026

Abstract

Fewer than 0.02 % of the proteins deposited in UniProt carry experimentally validated functional annotations. Computational methods attempt to close this gap by transferring Gene Ontology (GO) knowledge from characterised proteins to unknown ones, yet the ecosystem of tools for doing so is fragmented across notebooks and one-off scripts, and most published benchmarks conflate training and test annotations through temporal leakage, inflating reported performance.

This doctoral thesis presents **PROTEA** (*PROtein funcTional annotation framEwork and plAtform*), designed and maintained by Francisco Miguel Pérez Canales, an open-source distributed platform that automates the full pipeline from raw protein sequence to structured GO predictions. PROTEA couples protein language model (PLM) embeddings from eight backends (including ESM-2, ESM-C, ProstT5, ProtT5, Ankh and ESM-3) with approximate k-nearest-neighbour search over 527,000 reviewed UniProt sequences and a contracts-first plugin architecture spanning seven code repositories, so that new backends, annotation sources and ranking methods can be added without modifying the core platform.

Prediction quality is improved by a LightGBM re-ranker trained on a 52-feature schema combining embedding distance, Needleman-Wunsch and Smith-Waterman alignment statistics, NCBI-taxonomy-based distance metrics, and GO-graph ancestry embeddings (anc2vec). Training follows a leakage-free temporal holdout over twelve consecutive GOA releases. Under this protocol, the re-ranker raises NK-BPO $F_{\max}$ from 0.431 to 0.599 relative to the embedding-only baseline, a gain that disappears almost entirely when temporal leakage is present, underscoring the importance of rigorous evaluation methodology.

The experimental work contributed to a team placement of second in CAFA 6, validating the approach in a blinded international benchmark. Three pre-built containers were submitted to the LAFA public benchmark, demonstrating deployment across development, cloud, HPC (Apptainer) and airgapped environments.

The doctoral contribution is threefold: (1) PROTEA as a reproducible, operationally auditable platform; (2) a leakage-free reranker pipeline that quantifies the true lift of feature engineering over PLM embeddings; (3) the LAFA v2 method submission as an independently verifiable performance reference.

Keywords: protein function prediction, gene ontology, protein language models, sequence embeddings, nearest-neighbour search, learning-to-rank, LightGBM, distributed systems, reproducibility, container deployment, bioinformatics infrastructure.

Acknowledgements

[To be written.]

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

The pace at which biological sequence data accumulates has long outstripped our capacity to characterise proteins experimentally. As of early 2026, Universal Protein Resource (UniProt) contains more than 250 million sequences, yet fewer than 0.02 % carry experimentally validated functional annotations [50]. The gap is not merely quantitative: the proteins that remain unannotated are disproportionately found in under-studied organisms, precisely where novel biology, and potential therapeutic targets, are most likely to reside. Every new genome sequencing campaign deepens the shortfall, because sequencing throughput scales with Moore's law while experimental characterisation throughput scales with laboratory headcount.

Protein function, in the sense used throughout this thesis, is represented by the Gene Ontology (GO): a structured, species-independent vocabulary of molecular functions, biological processes, and cellular components. GO terms are organised as directed acyclic graphs (DAGs), so that annotating a protein with a specific term entails an implicit annotation with every ancestral term. This structure is both a strength and a complication. It is a strength because it allows partial knowledge to be expressed cleanly: a protein may be known to participate in a biological process without the precise molecular mechanism being resolved. It is a complication because predictions must respect the DAG constraints, and performance metrics must account for the hierarchical nature of the label space.

Computational function prediction attempts to bridge the annotation gap by transferring knowledge from characterised proteins to unknown ones. Classical approaches based on sequence similarity (BLAST, profile Hidden Markov Models (HMMs)) remain widely used, but their accuracy degrades for distantly related sequences where the evolutionary signal is too weak for reliable alignment. Sequence-based methods reach a natural ceiling at roughly 30 % pairwise identity: below this threshold, the "twilight zone" of sequence similarity, alignment scores lose discriminative power and the majority of the evolutionary space that separates distant homologues becomes inaccessible. Structure-based approaches, including Foldseek and derivatives of AlphaFold2 embeddings, extend the reach further by exploiting the fact that protein structure is more conserved than sequence. However, structure-based methods depend on high-quality structure predictions and introduce additional computational cost that is prohibitive at the scale of hundreds of thousands of reference sequences.

Over the last five years, Protein Language Models (pLMs) trained on hundreds of millions of sequences have produced dense vector representations that capture structural and functional signal well beyond what classical alignment can detect. The key insight is that a large transformer trained on a protein sequence corpus with a masked-language objective learns, in its internal representations, a compressed model of protein physics that correlates with structure, evolutionary distance, and function. ESM-2 [34] and its successors, ProtT5 [14], Ankh [15], and ESM-C have demonstrated that a single forward pass through such a model produces embeddings whose nearest neighbours in the representation space correlate strongly with functional similarity. A generation of structure-conditioned models (ESM-3, ProstT5) has begun to integrate sequence and structure into a single representation, opening a new operating point in the accuracy/coverage trade-off.

The practical consequence is that embedding-based annotation transfer, sometimes called PLM-KNN (protein language model nearest-neighbour), has emerged as one of the most effective families of methods in the CAFA benchmarks. The approach is straightforward: embed the query sequence and every sequence in a functionally characterised reference set using the same PLM, retrieve the k nearest neighbours by cosine distance, and aggregate their annotations weighted by similarity. Despite its simplicity, this approach is competitive with methods that incorporate structural priors, GO-DAG graph convolutions, or hand-crafted sequence features.

Each new PLM unlocks new predictive power, but also introduces a new dependency, a new tokeniser, a new GPU memory footprint, and a new failure mode. Comparing PLMs at scale, on a fair benchmark, with reproducible numbers, is itself a research engineering problem. It is not sufficient to test a single model on a single snapshot of the annotation database: the useful questions involve how performance varies across ontology aspects (Biological Process, Molecular Function, Cellular Component), across knowledge categories (sequences with no prior annotations, limited annotations, or partial annotations), and across sequence identity regimes. Answering these questions requires infrastructure that can embed hundreds of thousands of reference sequences, perform fast nearest-neighbour search, score predictions against a carefully constructed ground truth, and log the provenance of every intermediate artefact.

The feature-engineering question adds a further layer of complexity. Raw embedding distance is a useful but noisy signal: two sequences may be close in the PLM embedding space for reasons that do not translate into shared GO annotations. Pairwise alignment scores (Needleman-Wunsch global alignment, Smith-Waterman local alignment) complement the embedding distance with a signal grounded in evolutionary substitution models. Taxonomic distance, computed from the NCBI taxonomy tree, captures the evolutionary relatedness of the source organism, which is predictive of how well annotations transfer. GO-graph ancestry embeddings (anc2vec) encode the structural position of candidate terms in the ontology DAG, adding a signal that is orthogonal to both sequence and taxonomy. The question of whether these signals, combined in a learned ranking model, improve over the embedding-only baseline in a leakage-free setting is a central empirical question of this thesis.

A second, equally important problem is evaluation rigour. Most published benchmarks in protein function prediction conflate training and test annotations by drawing both from the same annotation snapshot, which introduces temporal leakage: the model implicitly benefits from annotations that post-date its training cut-off. When an embedding model is trained on Swiss-Prot sequences and subsequently evaluated on the same snapshot, any annotation that a neighbour carries is a legitimate training signal. The problem compounds when the reranker is also trained on the same snapshot, because the re-ranker's features (alignment scores, taxonomy proximity, GO-graph ancestors) are computed against the very annotations on which it is also scored. The result is an inflated picture of generalisation. Published leakage-corrupted Fmax numbers are substantially higher than numbers obtained under a rigorous holdout, and the delta is large enough that it matters to how one ranks and compares methods.

A meaningful benchmark must follow the temporal holdout protocol established by the Critical Assessment of Protein Function Annotation (CAFA) [8]: fix a reference annotation set at time t_0, make predictions for sequences with no or limited annotations at that date, and evaluate only against annotations that accrued between t_0 and t_1. This protocol isolates the genuine predictive power of the method from the passive accumulation of annotations in the reference database. It is the protocol that made CAFA a trusted community benchmark, and it is the protocol that PROTEA implements end-to-end, including for the re-ranker training pipeline.

The CAFA 6 challenge, whose results were released in late 2025, provided an independent blind validation of methods built around these principles. The research team in which the author of this thesis served as technical lead achieved a second-place ranking, validating the embedding-based annotation transfer approach at international scale and under the strictest possible evaluation conditions. The CAFA result does not belong to PROTEA as a platform: PROTEA is the post-challenge productisation. The result belongs to the research, and it is cited here as evidence that the methodology developed in this thesis is competitive with the best publicly evaluated alternatives.

1.1.1 Research Engineering as a First-Class Output

Protein function prediction has a reproducibility crisis that is distinct from, but related to, the leakage problem. Published methods are frequently described at a level of detail that does not permit independent implementation, let alone replication of the reported numbers. Source code, when released, is often a collection of single-use scripts tied to a specific dataset and a specific version of a database that may no longer be available. The result is a literature in which each method is evaluated on its own best-case benchmark, making cross-method comparisons unreliable.

The problem is structural, not a consequence of individual carelessness. Producing a reproducible, deployable research artefact is a substantial engineering investment, and it is not rewarded by the current publication incentive structure. Methods papers are evaluated on their headline numbers, not on whether a third party can independently verify those numbers on a fresh annotation snapshot three years later. The consequence is that the community's shared knowledge is encoded primarily in published results rather than in executable, queryable systems.

This thesis takes the position that building such a system is itself a scientific contribution. PROTEA is not merely a vehicle for producing numbers: it is a platform whose design embodies a set of commitments about reproducibility, auditability, and extensibility that have consequences for how protein function prediction research can be conducted going forward. The ADR model, the versioned data schema, the plugin contracts, and the leakage-free evaluation pipeline are not implementation details. They are the mechanism by which the scientific claims of the thesis are made independently verifiable.

1.1.2 The Engineering Challenge

The scientific problem sketched above has a shadow that receives less attention in the literature: the problem of building infrastructure that can support this kind of research sustainably. A typical research implementation of embedding-based annotation transfer consists of a notebook that embeds a set of sequences, a script that performs a cosine similarity search, and a second script that scores the results. This workflow is adequate for a one-off experiment but fails to scale in several dimensions.

First, it does not compose. When a new PLM is released, the entire embedding step must be repeated, typically by copy-pasting the script, introducing version drift and making comparisons unreliable. When a new annotation source (QuickGO, UniProt) must be added, a new loader is written outside the framework. When the evaluation protocol must be changed (from a single snapshot to a temporal split), the scoring logic must be rewritten from scratch.

Second, it does not audit. A one-off script produces a prediction file but does not record the ontology version, the annotation release, the embedding model configuration, the nearest-neighbour search parameters, or the feature-engineering decisions. Reproducing the result six months later requires either perfect memory or a trail of ad-hoc log files. Neither is reliable.

Third, it does not deploy. A notebook is not a method that can be submitted to a public benchmark, run in an HPC cluster, or handed to a collaborator without significant additional work. The gap between "it runs on my machine" and "it is deployable as an independently verifiable artefact" is routinely underestimated.

PROTEA was designed to address all three dimensions simultaneously. As a method, it operationalises embedding-based annotation transfer with selective learning-to-rank refinement, following a leakage-free temporal protocol throughout. As a platform, it provides the extensible foundation that lets new backends, annotation sources, and ranking methods be plugged in without ad-hoc glue. As a deployment story, it ships pre-built containers that run the same canonical pipeline on a developer workstation, in a cloud cluster, on an HPC system via Apptainer, or in an airgapped environment.

The architectural answer to composability is the operation abstraction: every unit of domain logic is a Python class with a name and an `execute` method, registered at startup in an `OperationRegistry`. Adding a new step to the pipeline means writing a new operation class, not modifying any existing one. Workers are stateless with respect to domain logic and route messages purely by name. This design makes the platform horizontally extensible without touching the core, and it makes every operation independently testable.

The architectural answer to auditability is the versioned data model combined with the `Job.findings` narrative. Every job writes its inputs (ontology version, annotation release, embedding configuration, search parameters) to a structured database record before execution begins. Findings written during execution are attached to the job record as structured observations. The result is that the database is not only a store of predictions but a journal of the research: every number in the evaluation chapter can be traced to a specific job, with the configuration and the observations that produced it.

The architectural answer to deployability is the contracts-first plugin split. Because the inference path (`protea-method`) and the embedding backends (`protea-backends`) carry no dependency on the platform's ORM, queue, or API machinery, they can be bundled into a standalone container with a FASTA file and a reference set of parquet embeddings. This is the artefact submitted to the LAFA benchmark. The same artefact runs on an HPC cluster with Apptainer without modification.

The platform is deliberately built around the assumption that the canonical pipeline of today will be obsolete tomorrow. ESM-2 was state-of-the-art in 2022; ESM-C and ESM-3 succeeded it; the next generation is already in preparation. What must remain stable across this evolution is the contract between layers, the trace of every run, and the pathway from a FASTA file to a ranked, interpretable Gene Ontology prediction. The plugin architecture ensures that replacing the embedding backend requires only a new class implementing the `EmbeddingBackend` ABC, not a rewrite of the pipeline. The versioned data model ensures that embeddings computed with the old backend are preserved alongside those computed with the new one, enabling controlled comparisons rather than overwrites.

Thesis statement. PROTEA demonstrates that a contracts-first, plugin-driven platform for protein function annotation can simultaneously deliver a reproducible, leakage-free evaluation methodology, quantifiable gains from feature-engineered re-ranking over PLM-embedding baselines, and independently verifiable deployment as a public benchmark submission, together constituting a reusable infrastructure for the next generation of annotation research.

1.2 Research Questions

The thesis addresses the following questions, each of which corresponds to a distinct dimension of the problem. The questions are deliberately ordered from the most scientific (what does embedding-based transfer achieve?) to the most architectural (how should a platform that supports this science be built?). This ordering reflects the view that the engineering decisions are not independent of the scientific ones: the choice to plug PLM backends rather than hard-code them is not aesthetic but empirical, motivated by the expectation that the answer to RQ1 will change as PLMs evolve. Similarly, the choice to implement leakage-free evaluation as an integral part of the platform rather than as a post-hoc script is motivated by the expectation that RQ2 cannot be answered reliably unless the evaluation methodology is correct.

RQ1. Can embedding-based k-Nearest Neighbours (kNN) annotation transfer match or exceed alignment-based methods for Gene Ontology (GO) term prediction, particularly at low sequence identity, and how does the answer change as new PLMs replace older ones?

This question drives the choice to treat the PLM backend as a plugin rather than a fixed dependency: the answer must be re-evaluable without changing the platform.

RQ2. Do pairwise alignment statistics, taxonomic distance, ontology-graph ancestry features, and per-PLM PCA projections provide complementary signal that improves the ranking of candidate annotations once the embedding baseline is already strong?

This question motivates the feature-engineering layer and the learning-to-rank reranker. The relevant evidence is whether the 52-feature schema adds measurable lift on top of the embedding-only baseline under a leakage-free evaluation, and whether that lift is robust across ontology aspects and knowledge categories.

RQ3. How can a reproducible, scalable annotation pipeline be designed so that researchers can re-run analyses as ontologies and databases are updated, and so that the pipeline itself can be deployed in development, cloud, on-premise HPC, and airgapped environments without code branching?

This question is architectural. The answer involves the operation abstraction, the plugin contract system, the versioned data model, and the container deployment matrix, all of which are developed in chapter 4 and chapter 5.

RQ4. How can the journey of a research platform, with its dead ends and pivots, be captured operationally so that the canonical pipeline is justified by an auditable trace rather than by post-hoc rationalisation?

This question motivates the `Job.findings` narrative model and the Architecture Decision Record (ADR) index, which record the why of every non-obvious choice alongside the what of every computed result.

1.3 Scope and Limitations

This thesis is explicitly a research engineering work, not a biology work. The application domain is protein function prediction, but the author's expertise is in the design and implementation of reproducible machine-learning pipelines, not in the biochemistry of protein function. Where biological claims appear, they are grounded in the existing literature and in the GO ontology itself, not in original biological analysis. The thesis does not propose new protein language models, new graph neural network architectures, or new GO terms. It proposes a platform within which such contributions can be developed, evaluated, and deployed.

The evaluation is conducted on a single hardware configuration (one workstation with a 16 GB GPU). Large-scale multi-GPU experiments, distributed training of the reranker, and evaluation at full UniProt scale are out of scope. The reference set of approximately 527,000 Swiss-Prot sequences covers the reviewed, experimentally curated portion of UniProt. Extending PROTEA to TrEMBL-scale (hundreds of millions of sequences) would require replacing the NumPy-based exact nearest-neighbour search with a distributed approximate search backend; FAISS provides the first step in that direction, and the platform is designed to accommodate it.

The thesis covers three LAFA benchmark submissions, which establish the deployability of the platform and provide an independently verifiable performance reference. Improving the LAFA ranking is not an objective of the current thesis: the submission demonstrates that the same code path that runs on the development workstation can be packaged as a benchmark-ready container without modification.

Finally, this thesis does not propose a new PLM pre-training scheme. The embedding models used (ESM-2, ESM-C, ProstT5, ProtT5, Ankh) are third-party models accessed via their public APIs or weights. PROTEA's contribution is to the systems layer: the platform that orchestrates their use, the evaluation harness that measures their output fairly, and the re-ranker that combines their representations with additional features to improve prediction quality. This is a deliberate positioning: the PLM space is advancing rapidly, and a system designed to use the best available model at any given time is more durable than one optimised for a specific model version. The author's expertise is in research engineering and applied machine learning, not in neural network architecture research, and the system design reflects this honestly.

1.4 Contributions

The main contributions of this work are:

- **PROTEA**: an open-source distributed platform implementing the full annotation pipeline, from FASTA upload to structured GO predictions with optional feature engineering and selective learning-to-rank re-ranking. PROTEA is designed, implemented, and maintained solely by the author of this thesis, and is available under an open-source licence at the address given in chapter 5.

- A **plugin architecture** based on Python `entry_points`, organised across seven code repositories (core, contracts, method, sources, backends, runners, and the offline reranker lab), that lets new annotation sources, runners, and PLM backends be added without modifying the core platform. The contracts package (`protea-contracts`) defines the extension surface as a SemVer-versioned public artefact independent of the platform stack.

- A **versioned data model** for ontology snapshots, annotation sets, embedding configurations, datasets, re-ranker models, and experiment runs, with provenance complete enough to exactly replay any past prediction. The model is encoded as a sequence of Alembic migrations under version control.

- A **benchmark of eight PLM backends** (ESM-2 in three sizes, ESM-C, ProstT5, ProtT5, Ankh, ESM-3 component encoder) on a common evaluation surface, with per-aspect, per-category, and per-backend numbers obtained under a leakage-free temporal holdout covering twelve consecutive GOA releases.

- A **feature engineering layer** that augments embedding similarity with Needleman-Wunsch / Smith-Waterman alignment scores, NCBI-taxonomy-based distance metrics, GO-graph ancestry embeddings (anc2vec), and per-PLM PCA projections, organised as a registry of approximately 52 features. The feature schema fingerprint ensures that any stored re-ranker model is automatically invalidated if the feature definition changes.

- A **selective learning-to-rank re-ranker** (LightGBM, trained per category and per aspect) that falls back to the embedding-only baseline whenever re-ranking would degrade performance for the current cell. Under the leakage-free protocol, the re-ranker raises NK-BPO $F_{\max}$ from 0.431 to 0.599, a gain of 0.168

absolute that disappears almost entirely when temporal leakage is introduced, underscoring the importance of the evaluation methodology.

- A **multi-target deployment story** (development `docker compose`, cloud Helm/Compose, HPC Apptainer, airgapped bundle) covering the same canonical pipeline without code branching. The deployment matrix was validated by submitting three pre-built containers to the LAFA public benchmark.

- An **operational narrative model** (`Job.findings`, `ExperimentRun`, `JobEvent` comments) that captures the why of every run and provides the audit trace from which the evaluation chapter is distilled. Complementing this, an index of 31 Architecture Decision Records (D1 to D31) in `protea-core/docs/source/adr/` provides a curated history of non-obvious design choices.

- An **empirical evaluation** comparing PROTEA predictions against baseline methods on held-out Gene Ontology Annotation (GOA) splits under a leakage-free temporal holdout (GOA 220 $\rightarrow$ 229), demonstrating that the 52-feature LightGBM re-ranker raises NK-BPO $F_{\max}$ from 0.431 to 0.599 relative to the embedding-only baseline.

- A **contribution to CAFA 6** as part of a research team, achieving a second-place team ranking in the international challenge, and three pre-built container submissions to the LAFA public benchmark as independently verifiable performance references.

1.5 Thesis Structure

The remainder of this document is organised as follows. The chapters are sequenced to build understanding progressively: biological background and related work provide the necessary context before the system design and implementation are presented, and the evaluation chapter can then be read against a concrete understanding of what the system does and why it was designed that way.

Chapter 2 introduces the biological concepts underpinning the work. It covers the Gene Ontology in depth (aspects, DAG structure, evidence codes, the role of IEA (Inferred from Electronic Annotation) evidence codes and why IEA annotations are excluded from the evaluation ground truth), surveys the modern landscape of protein language models from ProtBERT and ProtT5 through ESM-2 to ESM-C and ESM-3, and situates PLM-based annotation transfer within the broader landscape of embedding-based machine learning. Readers with a strong background in either biology or machine learning may read this chapter selectively.

Chapter 3 surveys existing computational approaches to protein function prediction up to 2025. It covers the CAFA challenge history and winning approaches, embedding-based methods (TransFew, FunBind, GoCurator), GO-DAG machine learning methods (GeOKG and related work), and the LAFA benchmark ecosystem. It situates PROTEA's contributions relative to this landscape, identifying both the deltas and the complementary strengths, and is explicit about the cases where PROTEA chooses depth over breadth.

Chapter 4 presents the system architecture of PROTEA at the level of design decisions. It develops the five core requirements (Reproducibility, Scalability, Separation of Concerns, Observability, Accessibility), the operation abstraction and the OperationRegistry, the plugin contract system and the contracts-first split across seven repositories, the job lifecycle and the ten-queue RabbitMQ topology, the versioned data model, the embedding and prediction pipelines, the ArtifactStore, and the public REST API with its seventeen routers. The ADR index D1 to D31 cross-references the major design decisions to this narrative.

Chapter 5 describes how the design materialises across the seven repositories. It covers the technology stack, the database schema and Alembic migrations, the 33-operation catalogue, feature engineering and the schema fingerprint mechanism, approximate nearest-neighbour search with NumPy and FAISS, the LightGBM reranker promotion pipeline, the Next.js frontend, and the testing strategy across unit, integration, and end-to-end levels.

Chapter 6 reports the experimental results. It defines the NK/LK/PK evaluation categories and the metrics ($F_{\max}$, coverage, AuPRC), establishes the leakage-free temporal holdout over fifteen GOA snapshots, presents per-aspect and per-category results for the eight PLM backends, traces the sequence of experiments from the KNN baseline to the LightGBM reranker, and includes a comparative study against eggNOG-mapper. The chapter also documents the impact of temporal leakage on reported performance and quantifies the gap between leakage-corrupted and leakage-free numbers, motivating the evaluation design choices made throughout.

Chapter 7 summarises the findings, answers the four research questions, discusses limitations (single workstation evaluation, GO ontology drift, PLM version churn), and outlines future directions including post-defense platform extensions, the possibility of integrating R-GCN graph convolution over the GO-DAG, and the open research agenda. Readers who prefer to begin with the experimental results before the architectural narrative may read chapter 6 independently; the evaluation chapter is written to be self-contained with respect to the evaluation protocol, and forward-references to earlier chapters are limited to definitions that cannot be assumed without background.

Chapter 2

Biological Background

This chapter provides the biological foundation required to understand the problem that PROTEA addresses. Readers with a background in molecular biology may skip directly to chapter 3.

2.1 Proteins: Structure and Function

Proteins are macromolecules composed of chains of amino acids linked by peptide bonds. The linear sequence of amino acids, encoded by the genome, is referred to as the *primary structure*. This sequence folds, driven by thermodynamic forces, into a specific three-dimensional conformation that determines the protein's biological activity.

Protein functions are extraordinarily diverse: enzymes catalyse chemical reactions, structural proteins provide mechanical support, signalling proteins relay information between cells, and transport proteins shuttle molecules across membranes. Understanding *what* a protein does (its molecular function, the biological process it participates in, and the cellular compartment where it operates) is fundamental to biology and medicine.

2.1.1 Amino Acids and the Sequence Space

Twenty standard amino acids, differing in their side-chain chemistry, constitute the alphabet of protein sequences. A protein of n residues can in principle take 20^n distinct sequences, an astronomically large space. Yet the sequences that fold into stable, functional structures represent a tiny, structured subset of this space, one in which similar sequences tend to adopt similar folds and perform similar functions.

2.1.2 Sequence Identity and Homology

Two proteins are said to be *homologous* if they share a common evolutionary ancestor. Homology is typically inferred from sequence similarity: proteins with high pairwise sequence identity are almost always homologous and usually perform the same function. As sequence identity drops below roughly 30 %, however, the relationship between similarity and function becomes ambiguous: proteins may be structurally similar yet functionally diverged, or functionally convergent with unrelated folds.

2.2 Protein Databases

2.2.1 UniProt

UniProt is the central repository for protein sequence and functional information [50]. It is divided into two sections:

Swiss Protein database (reviewed section of UniProt) (Swiss-Prot) A manually curated, high-quality database. Each entry has been reviewed by expert annotators and contains experimental or computationally inferred functional annotations, with explicit evidence codes.

Translated EMBL Nucleotide Sequence Data Library (TrEMBL) An automatically annotated, computationally analysed database derived from genome translations. It is several orders of magnitude larger than Swiss-Prot but of lower average annotation quality.

Each protein in UniProt is identified by a stable accession number. Canonical isoforms are represented by a bare accession (e.g. `P12345`); alternative splicing variants are denoted `P12345-2`, `P12345-3`, and so on.

2.2.2 The Annotation Bottleneck

The asymmetry between Swiss-Prot and TrEMBL defines the central problem that computational annotation methods attempt to solve. As of the 2023 release, UniProt contains approximately 250 million sequences in TrEMBL against fewer than 600,000 manually reviewed entries in Swiss-Prot [50]. Experimental characterisation of a single protein can require years of biochemical work; sequencing a genome is now routine. The result is that the vast majority of known protein sequences carry no experimentally validated functional annotation whatsoever, and the gap widens with every large-scale sequencing project.

Automated function prediction methods exist precisely to bridge this gap. Their practical value is measured by how reliably they can annotate TrEMBL proteins at Swiss-Prot quality, using the experimentally annotated proteins as a labelled reference. This framing, inference from a characterised reference set to an uncharacterised query, is the operational setting throughout this thesis.

2.2.3 Gene Ontology Annotation Database

The GOA database aggregates GO annotations for UniProt proteins from multiple contributing databases (UniProt-KB, IntAct, Ensembl, and others) and releases them as versioned flat files [19]. Each release is timestamped and carries a release number; releases are published at regular intervals (roughly monthly). The versioned structure is operationally important: PROTEA imports GOA releases as immutable `OntologySnapshot` objects and associates every annotation set and evaluation target with a specific release pair, enabling reproducible leakage-free evaluation (chapter 6).

IEA annotations make up the large majority of records in any GOA release. By convention, benchmark evaluations (including CAFA) exclude IEA annotations from the ground-truth set and treat only experimentally supported codes as valid prediction targets. PROTEA follows this convention: the training reference set and the evaluation target set both filter to experimental evidence codes, and the `EvidenceFilter` component enforces this policy at ingestion time.

2.2.4 Protein Data Bank

The Protein Data Bank (PDB) archives three-dimensional structures of biological macromolecules determined experimentally by X-ray crystallography, cryo-electron microscopy, or NMR [4]. Although PROTEA operates primarily on sequences, PDB structures are an important source of ground-truth functional information and are used indirectly through UniProt cross-references.

2.3 The Gene Ontology

2.3.1 Overview

The GO is a standardised, species-independent controlled vocabulary for describing gene product attributes [3, 19]. It is structured as three independent Directed Acyclic Graphs (DAGs), one for each of the three top-level *aspects*:

Molecular Function (MF) (GO:0003674) The biochemical activity of the gene product, independent of where or when it occurs (e.g. *ATP binding*, *serine-type endopeptidase activity*).

Biological Process (BP) (GO:0008150) The larger biological programme to which the activity contributes (e.g. *apoptotic process*, *DNA repair*).

Cellular Component (CC) (GO:0005575) The place in the cell where the gene product is active (e.g. *nucleus*, *plasma membrane*).

Terms within each aspect are connected by *is-a* and *part-of* relationships, forming a hierarchy from broad root terms to highly specific leaves. A protein annotated with a term is implicitly annotated with all of its ancestors up to the root: the *true path rule*.

The depth and specificity of terms vary substantially across the ontology. Root nodes such as GO:0003674 (Molecular Function) are trivially true for every protein and carry no discriminative information; deep leaf terms such as GO:0004364 (glutathione transferase activity) are experimentally supported for only a handful of well-characterised proteins. This specificity gradient is captured by the *information content* (IC) of a term, defined as the negative log of the fraction of annotated proteins that carry it. Prediction quality metrics that weight by IC (such as $S_{\min}$, used in CAFA) reward correct prediction of specific terms more than generic ones. PROTEA's nearest-neighbour transfer propagates annotations at all specificity levels and reports predictions over the full ontology; the re-ranker described in chapter 4 operates per-term and per-aspect, allowing it to learn specificity-dependent calibration.

2.3.2 Evidence Codes

Every GO annotation carries an evidence code that indicates the type of support for the association. The GO Consortium defines three broad categories:

Experimental Annotations derived from direct experimental assays. Representative codes: `EXP` (Experiment), `IDA` (Inferred from Direct Assay), `IMP` (Inferred from Mutant Phenotype), `IGI` (Inferred from Genetic Interaction), and `IPI` (Inferred from Physical Interaction). These constitute the gold-standard ground truth for benchmark evaluation.

Computational Annotations generated without direct experimental support. `ISS` (Inferred from Sequence or Structural Similarity) and `IEA` (Inferred from Electronic Annotation) are the most prevalent. `IEA` annotations are produced automatically by tools such as UniProt rule-based systems and InterPro domain mappings; they are numerous but carry no experimental validation.

Author-stated Annotations from author statements in publications without direct assay evidence: `TAS` (Traceable Author Statement) and `NAS` (Non-traceable Author Statement).

The distinction between experimental and computational codes is operationally consequential for function prediction benchmarks. The CAFA protocol, and PROTEA's evaluation design, treat only experimental-evidence annotations as valid prediction targets. `IEA` annotations are excluded from both the training reference set and the evaluation ground truth: including them would allow a method to "predict" machine-generated labels with machine-generated features, which does not constitute meaningful generalisation. The `EvidenceFilter` component in PROTEA's ingestion pipeline enforces this boundary at import time.

2.3.3 Ontology Versioning

The GO is actively maintained and released regularly in Open Biological and Biomedical Ontologies (OBO) format. New terms are added, obsolete terms are deprecated, and relationships are refined with each release. Reproducible research therefore requires that analyses record which ontology version was used. PROTEA stores each imported release as an `OntologySnapshot` and associates all annotation sets and predictions with a specific snapshot.

2.4 Sequence Alignment

Pairwise sequence alignment is the classical approach to measuring sequence similarity. Two algorithms are central to this thesis:

Needleman-Wunsch (Needleman-Wunsch (NW)) A global alignment algorithm that aligns two sequences end-to-end, maximising a cumulative substitution score minus gap penalties [37]. It is suitable for comparing proteins of similar length.

Smith-Waterman (Smith-Waterman (SW)) A local alignment algorithm that finds the highest-scoring contiguous aligned region, ignoring the tails of each sequence [46]. It is more appropriate when one sequence is a domain or fragment of the other.

Both algorithms use a *substitution matrix* (typically BLOSUM62 for protein sequences [24]) to score residue substitutions based on their observed frequency in alignments of related proteins.

2.5 Protein Language Models and Embeddings

PLMs are neural networks trained on large corpora of protein sequences using self-supervised objectives borrowed from natural language processing. Models trained with a masked language modelling objective on hundreds of millions of sequences develop internal representations that capture evolutionary, structural, and functional information without explicit supervision.

A protein sequence can be passed through such a model to obtain a dense vector (embedding) that encodes its properties in a continuous latent space. Proteins with similar functions tend to cluster together in this space even when their sequences have diverged below the homology detection threshold of alignment-based tools, which is what makes embeddings attractive for annotation transfer.

2.5.1 The 2020 to 2024 Generation: Sequence-Only Encoders

The first generation of practical pLMs for function prediction was based on Transformer encoders trained with a masked language modelling objective on sequence-only corpora.

ESM-1b [43] and the **ESM-2** family [34] (Meta AI) span eight model sizes from 8M to 15B parameters, all trained on UniRef50 / UniRef90. Each scale increment is accompanied by a measurable gain on structure prediction and remote-homology detection benchmarks. PROTEA evaluates ESM-2 at three operating points (650M, 3B, 15B) to characterise the accuracy/throughput trade-off.

ProtTrans [14] provides a suite of encoder models (ProtBERT, ProtAlbert, ProtXLNet, ProtT5) trained on Big Fantastic Database (BFD) and UniRef100 using distributed HPC resources. ProtT5-XL achieves the best per-residue and per-protein accuracy among the ProtTrans variants.

Ankh [15] is an encoder-decoder model trained on UniRef50 with a focused objective and aggressive vocabulary tuning. It reports state-of-the-art accuracy on several downstream tasks at a fraction of the parameter count of the largest ESM-2 variants.

2.5.2 The 2024 to 2025 Generation: Structure-Aware and Distilled Models

Three trends shape the most recent generation: integration of structural signal during training, further parameter efficiency, and sequence/structure multi-modal representations.

ProstT5 [23] extends ProtT5 with a bilingual sequence-to-3Di mapping (3Di is the Foldseek structural alphabet derived from predicted three-dimensional structures), letting the model encode structural information without an explicit 3D input. The same encoder serves both sequence-only and structure-aware downstream tasks.

ESM-3 [22] is a multi-modal generative model over sequence, structure and function tokens, reaching parameter scales up to 98B. For embedding-based downstream applications PROTEA uses the lighter component encoder (ESM-3c), which provides per-residue and per-protein embeddings comparable in interface to the ESM-2 family.

ESM-C [16] (released in late 2024 by EvolutionaryScale) is a family of efficient sequence-only encoders distilled from larger ESM-3 variants. ESM-C 300M and 600M deliver embeddings competitive with ESM-2 3B at a fraction of the inference cost, making it the smallest backend at which PROTEA achieves strong CAFA-style performance.

2.5.3 Inference Cost and the Operating-Point Question

Each PLM occupies a distinct operating point on the accuracy/throughput trade-off. ESM-2 15B produces the strongest embeddings but requires several days on a single A100 to embed 500,000 sequences; ESM-C 300M produces embeddings of comparable downstream quality on the same hardware in under a day. Choosing a backend is therefore a deployment decision more than a modelling one, and a platform that operationalises PLM-based annotation at scale must abstract the choice cleanly. Chapter 4 describes how PROTEA implements that abstraction through the `EmbeddingBackend` plugin contract.

2.6 Taxonomic Classification

Biological organisms are classified into a hierarchy (the NCBI taxonomy) that reflects their evolutionary relationships [45]. Each node is assigned a numeric *taxon ID*. The distance between two organisms can be defined operationally as the number of edges between them through their Lowest Common Ancestor (LCA) in the taxonomy tree.

Taxonomic distance is a useful co-variate for function prediction: a protein from a closely related organism is more likely to perform the same function than one from a distant lineage, even when embedding similarity is comparable. UniProt records the source organism for every sequence, so taxon IDs are available for all reference and query proteins. PROTEA computes taxonomic LCA distance between each query and every candidate neighbour as one of the features supplied to the re-ranking model (section 5.5), allowing the re-ranker to discount hits whose embedding similarity is high but whose evolutionary distance suggests the function may have diverged.

Chapter 3

Related Work

Automated protein function prediction has been an active research area for more than two decades. Methods can be grouped into five broad families: (i) sequence alignment transfer, (ii) profile and domain matching, (iii) supervised deep learning, (iv) embedding-based nearest-neighbour transfer, and (v) multiple instance learning and interpretable models. Complementing these methodological threads are the community benchmarking challenges that have calibrated the field's progress, and a layer of genome-scale annotation platforms that operationalise whichever inference method is selected. This chapter surveys each family, identifies their limitations, and contextualises PROTEA within the landscape.

3.1 Alignment-Based Annotation Transfer

The foundational insight of alignment-based methods is that sequence similarity implies functional similarity. If a query protein shares high sequence identity with a characterised protein, one can transfer the known annotations to the query.

BLAST [1] remains the most widely deployed tool for this purpose. It uses a heuristic seed-and-extend strategy to find local alignments in sublinear time. `BLASTP` (protein-to-protein) identifies homologues in Swiss-Prot and transfers their GO terms to the query. This simple approach is accurate when identity exceeds roughly 50 % and degrades progressively as identity falls. Below the "twilight zone" of ≈30 % identity, alignment scores lose statistical power and functional inference becomes unreliable [44].

DIAMOND [7] offers BLAST-comparable sensitivity at orders-of-magnitude higher throughput, making it practical for genome-scale annotation. Its speed advantage comes from reduced-alphabet seeds and better SIMD utilisation; the fundamental twilight-zone limitation remains.

PSI-BLAST [2] extends the basic BLAST paradigm through an iterative cascade: hits from a first-round search are assembled into a position-specific scoring matrix (PSSM), and subsequent rounds use that PSSM rather than a single query sequence. The approach substantially extends sensitivity into the twilight zone but introduces false-positive accumulation in later iterations, particularly for proteins with multi-domain architectures. PSI-BLAST prefigures the profile HMM approach described in section 3.2: both methods exploit evolutionary conservation captured across multiple sequences rather than a single representative.

Annotation propagation rules add a further layer of nuance: not all GO terms are equally transferable. Terms annotated with `IEA` evidence codes can be propagated freely, while experimental annotations require explicit curation policies. The Critical Assessment of Functional Annotation (CAFA) community has established benchmark protocols that penalise propagation errors [55].

3.2 Profile and Domain Methods

To extend sensitivity below the twilight zone, profile methods compare a query against position-specific models built from multiple sequence alignments of protein families.

HMMER [12] implements profile HMMs for this purpose. A family is represented as a probabilistic model over residue frequencies at each position, capturing conserved and variable sites more faithfully than a single representative sequence. HMMER achieves greater sensitivity than BLAST at the cost of requiring pre-built profiles. The algorithm's expectation-maximisation training scales to millions of sequences, and the HMMER3 release replaced the earlier quadratic forward algorithm with an accelerated filter pipeline that achieves near-linear average complexity without measurable loss of sensitivity.

Pfam [36] is the canonical curated database of protein domain families represented as HMM profiles. Each Pfam entry covers a structurally or functionally coherent domain; GO term mappings from Pfam entries are transferred to any protein whose sequence matches the profile above a calibrated significance threshold. Pfam is now maintained inside the **InterPro** [5] consortium, which integrates over a dozen family and domain databases (Pfam, PRINTS, ProSite, TIGRFAM, and others) into a unified annotation resource. A protein is assigned InterPro entries whenever it matches any member database; GO terms are then inferred via curated mappings. InterProScan, the companion command-line tool, is widely used as an annotation step in genome pipelines. InterPro annotation is one of the most reliable computational methods available, but it is limited to protein families and domains that have been explicitly curated and modelled; novel folds with no detectable sequence relatives receive no annotation.

Cascade homology search [38] iteratively extends sensitivity by feeding BLAST hits into further BLAST searches (PSI-BLAST) or HMM construction, at the cost of introducing false positives in remote homology regimes. The fundamental ceiling shared by all profile methods is coverage: organisms with many lineage-specific proteins benefit little from profile transfer.

3.3 Supervised Deep Learning Approaches

Supervised methods learn classifiers, one per GO term of interest, from labelled protein-function pairs. Early approaches used HMMs or support vector machines on curated feature sets (amino acid composition, physicochemical properties, secondary structure predictions). More recent work uses deep neural networks applied directly to sequences or structures.

3.3.1 Classical Supervised Classifiers and Hierarchical Correction

Before deep learning displaced them, the dominant family of supervised methods trained independent binary classifiers (support vector machines or random forests) for each GO term of interest, using features derived from sequence composition, physicochemical indices, or BLAST-derived profiles [42]. The label set for each classifier was drawn from the GOA database at a fixed release, ensuring that the model sees only experimentally curated annotations.

A critical design choice in all hierarchical prediction systems is how to enforce the true-path rule: if a protein is predicted to have function t, it must also be predicted to have every ancestor of t in the GO DAG. Naive independent classifiers violate this rule; post-processing propagation steps (or hierarchical loss functions) are required to restore consistency. The $S_{\min}$ metric [26] penalises violations of this consistency at evaluation time: it measures the minimum semantic distance between the prediction set and the ground-truth set, weighted by the information content of each term. Systems that produce high $F_{\max}$ without attending to $S_{\min}$ tend to over-predict high-level, low-specificity terms; the dual optimisation forces a specificity-coverage trade-off that coarse classifiers handle poorly.

GOA-trained classifiers of this form were the baseline for CAFA 1 through

CAFA 3. They achieved reasonable $F_{\max}$ on the Biological Process aspect (where GO depth is moderate and training data abundant) but struggled on Molecular Function (high depth, sparse positive labels) and Cellular Component (where localization-based features are not easily extracted from sequence alone). The transition to deep learning and later to pLM embeddings was driven precisely by the failure of hand-crafted features to distinguish proteins at the GO leaf level.

3.3.2 Sequence-Based Deep Models

DeepGO [33] and its successor **DeepGOPlus** [31] train convolutional networks on sequence k-mers and combine the output with sequence similarity scores from BLAST. DeepGOPlus demonstrated that learned sequence features and alignment evidence are complementary: the two signals fuse into a more reliable predictor than either alone. Ontology axioms encoded as logical implications are used to regularise the prediction, a direction later extended in DeepGOZero [32], which performs zero-shot prediction for terms with no training examples by grounding predictions in GO axioms expressed in EL description logic.

TALE [6] (Transformer-based Annotation Learning and Extraction) adapts the pre-trained ESM-1b encoder with a multi-label task head and a consistency loss that enforces true-path rule compliance across the GO DAG. Its evaluation on CAFA3 targets showed that fine-tuning a protein language model encoder on a function prediction objective produces substantially stronger results than using frozen embeddings, particularly for the Molecular Function aspect where alignment cues are weakest.

NetGO [53] and its successor **NetGO 3** [54] augment sequence-based models with protein-protein interaction network features. NetGO 3 incorporates PLM embeddings alongside STRING network context, achieving state-of-the-art results in CAFA3 and subsequent benchmarks. The approach requires interaction data, which limits applicability for poorly studied organisms and environmental metagenomic sequences for which interactomes are unavailable.

TALE, **NetGO 3**, and related methods established the pattern of fusing pre-trained pLM representations with task-specific supervision. PROTEA extends this line by decoupling representation (the embedding backend) from inference (the kNN retrieval plus re-ranker), enabling the same pipeline to be evaluated across multiple PLM backends without retraining classifiers.

3.3.3 Structure-Based Deep Models

The release of **AlphaFold 2** [28] and the subsequent AlphaFold Protein Structure Database [51] covering over 200 million proteins has enabled structure-informed annotation at a scale previously impossible. Several groups have exploited these predicted structures as an additional evidence channel.

DeepFRI [20] is among the most influential structure-based annotation methods. It applies a graph convolutional network over protein contact maps derived from either experimental or predicted structures, and learns residue-level attention weights that can highlight catalytic and binding sites. DeepFRI outperforms sequence-only baselines for enzyme function prediction and its residue-level interpretability is a distinct advantage over sequence-level classifiers.

HoloProt [47] combines a surface-based molecular representation with a hierarchical encoder that captures both global shape and local physicochemical patterns. The global surface representation is more rotation-equivariant than contact-map GCNs and performs competitively on enzyme commission number prediction, though its advantage over pLM-based methods shrinks as model size increases.

AlphaFold-informed annotators more broadly are a growing subfield. Approaches range from embedding AlphaFold-predicted structures with Foldseek's 3Di structural alphabet [30] for structure-based nearest-neighbour retrieval, to fine-tuning structure encoders on GO labels. The core limitation of this line is computational: predicting a structure for every query protein incurs the cost of an AlphaFold forward pass, which is prohibitive for millions of metagenomic sequences on a modest compute budget. PROTEA defers structural integration to the roadmap; ProstT5 (described in section 3.4) offers a partial bridge by encoding structural signals implicitly through sequence-to-3Di translation without requiring an explicit structure prediction step.

A fundamental challenge shared by all supervised methods, whether sequence-based or structure-based, is the extreme imbalance and hierarchy of the GO label space. The number of proteins annotated with any specific GO term decreases rapidly as one descends the ontology DAG; for many leaf terms, fewer than ten training examples exist. Hierarchical loss functions and ontology-aware evaluation metrics ($F_{\max}$, AUPR) have been developed to address this [11].

3.4 Protein Language Model Representations

The advent of large-scale self-supervised pLMs has opened a qualitatively different avenue for function prediction. Models trained on hundreds of millions of sequences develop internal representations (embeddings) that capture evolutionary relationships, secondary and tertiary structure propensities, and functional sites, without any explicit structural or functional supervision.

3.4.1 ESM Family

ESM-1b [43] demonstrated that masked-language modelling objectives over 250 million protein sequences yield representations that recover contact maps and functional annotations without task-specific supervision. The **ESM-2** family [34] (Meta AI, published in *Science* 2023) substantially extends this scaling study, ranging from 8M to 15B parameters, all trained on UniRef50 with a masked-language modelling objective. The key finding of Lin et al. is that embedding quality on downstream tasks scales near-log-linearly with model size up to 3B parameters, after which returns diminish. ESM-2 embeddings have served as the de facto baseline representation in the function prediction literature since 2023, and they constitute one of PROTEA's primary backends.

Ankh [15] is an encoder-decoder model that matches or exceeds the ESM-2 3B baseline on multiple downstream tasks while using a fraction of the parameters, achieved through aggressive vocabulary tuning and a re-thought training recipe.

The 2024 to 2025 generation moves in three complementary directions. First, **ProstT5** [23] introduces structural awareness without requiring explicit 3D input by training a sequence-to-3Di translation on top of ProtT5, where 3Di is the structural alphabet introduced by Foldseek [30]. Second, **ESM-3** [22] unifies sequence, structure and function as multi-modal tokens within a single generative model up to 98B parameters; for embedding-based downstream applications, the lighter component encoder (ESM-3c) is the practical choice. Third, **ESM-C** [16] delivers efficient sequence-only encoders distilled from ESM-3, with the 300M and 600M variants reaching ESM-2 3B performance at a fraction of the inference cost. PROTEA evaluates all of these as plug-in backends within a single benchmarking framework.

3.4.2 ProtTrans Family

ProtTrans [14] (Elnaggar et al., *IEEE TPAMI* 2022) provides a suite of encoder models trained on BFD and UniRef100 using distributed HPC resources. The suite includes ProtBERT (BERT architecture), ProtAlbert (ALBERT architecture), ProtXLNet, and **ProtT5-XL**, the latter of which achieves the best per-residue and per-protein accuracy among the ProtTrans variants. ProtT5 is an encoder-decoder architecture derived from Google's T5 pre-trained on the T5 sequence-to-sequence objective applied to protein sequences; the encoder outputs serve as fixed-dimensional embeddings for downstream classification. ProtT5-XL-U50 (half-precision) is PROTEA's default alignment-free backend and the representation used in the LAFA container submission described in chapter 7.

Ankh [15] (Elnaggar et al., 2023) is an encoder-decoder model that matches or exceeds the ESM-2 3B baseline on multiple downstream tasks while using a fraction of the parameters. This efficiency gain is achieved through aggressive vocabulary tuning (a compact amino-acid tokeniser that reduces sequence length compared to character-level tokenisers) and a re-thought training recipe that emphasises biologically meaningful objectives. Ankh is an attractive backend for resource-constrained deployments; it is evaluated alongside ESM-2 and ProtT5 in chapter 6.

3.4.3 Structure-Aware Sequence Models

ProstT5 [23] (Heinzinger et al., *NAR Genomics and Bioinformatics* 2024) introduces structural awareness without requiring explicit 3D input. Starting from the ProtT5-XL encoder, ProstT5 is fine-tuned on a sequence-to-3Di translation task, where 3Di is the structural alphabet introduced by Foldseek [30] that encodes the local structural context of each residue as one of 20 discrete tokens derived from predicted inter-residue geometry. The key result is that ProstT5 embeddings carry richer structure-related signal than pure sequence models while remaining applicable to any protein that lacks an experimental or predicted structure, because the structural signal is distilled into the model weights during training rather than requiring structure at inference time. For PROTEA, ProstT5 constitutes a "structure-aware at no extra cost" backend option, evaluated in chapter 6.

3.5 Embedding-Based Nearest-Neighbour Methods

Annotation transfer via nearest neighbours is the inference strategy most naturally paired with fixed-dimensional embeddings. Given a set of reference proteins with known annotations and their embeddings, the k nearest reference neighbours of a query (by cosine or Euclidean distance) are retrieved and their annotations aggregated to form a prediction.

Littmann et al. (2021) [35] provided the first systematic demonstration that embedding-based annotation transfer outperforms BLAST-based transfer in the twilight zone and for uncharacterised orphan proteins. Using ESM-1b and ProtT5 embeddings, they showed that kNN retrieval from Swiss-Prot consistently recovers more correct GO terms than pairwise alignment for sequences below 50 % identity. This paper established the empirical foundation for the PLM-KNN paradigm that PROTEA operationalises at scale.

FANTASIA [39] (used extensively in the biology domain from which PROTEA's evaluation data derives) applies kNN transfer using ESM-2 embeddings for large-scale marine metagenome annotation. Its success in annotating proteins with no known homologues motivated the PROTEA project, and its design informed the annotation source plugin architecture in PROTEA.

Scalability is the central engineering challenge for embedding-based transfer. Swiss-Prot alone contains over 570,000 entries; computing exact cosine distances against all of them for a large query set requires $O(NQ)$ dot products, which becomes infeasible at scale. Approximate nearest-neighbour libraries such as FAISS [27] reduce this to $O(Q\sqrt{N})$ with minimal accuracy loss for IVFFlat indices. PROTEA uses FAISS with IVFFlat indexing (the hard constraint prohibiting pgvector for collections exceeding 500k vectors is documented in ADR D10); numpy is used for smaller reference sets to preserve exact retrieval.

Aggregation strategies further differentiate PLM-KNN methods. Simple majority voting over k neighbours suffers when neighbours have uneven confidence or when the query sits equidistant from proteins with conflicting annotations. Score-weighted aggregation, where each neighbour's GO terms are weighted by its distance-derived similarity score, generally outperforms majority voting. Mean-of-scores aggregation across multiple complementary sources (alignment, embedding, domain matching) is an ensemble strategy evaluated in chapter 6 and forms the basis of PROTEA's multi-source scoring architecture.

Backend comparison across PLM models has received limited systematic treatment in the literature. Most published KNN systems fix a single embedding model and report results for that model only, making cross-model comparisons difficult due to differences in evaluation sets and splitting strategies. PROTEA addresses this gap by holding the inference pipeline fixed (same FAISS index structure, same reference corpus, same aggregation function) and varying only the embedding backend, producing a controlled comparison across ESM-2 (650M), ProtT5, Ankh, ProstT5, and ESM-C (600M). This controlled design is a methodological contribution independent of the performance numbers; chapter 6 reports the resulting backend ablation study.

3.6 Ensemble Methods and Learning-to-Rank

The observation that alignment-based, profile-based, and embedding-based signals are complementary motivates ensemble approaches that combine multiple scores for the same (protein, GO term) pair.

Score fusion is the simplest ensemble strategy: predictions from multiple independent methods are combined by taking the mean (or weighted mean) of their confidence scores, using the resulting composite as the final prediction. The assumption is that individual methods have uncorrelated errors, so averaging reduces variance. This strategy is used by PROTEA's multi-source aggregation layer and is the baseline against which the re-ranker is compared in chapter 6.

Learning-to-rank with gradient boosted trees is a natural next step. Rather than a fixed weighting scheme, a re-ranker learns from labelled examples which feature combinations are most predictive of a correct annotation. **LightGBM** [29] is the dominant gradient boosting framework for tabular re-ranking tasks, offering fast training, native handling of categorical features, and strong regularisation. PROTEA trains per-aspect LightGBM models whose feature set includes: cosine similarity to each neighbour, alignment bit-score (when available), ontology-aware GO term embeddings from anc2vec [13], information-content of the predicted term, and taxon-distance between query and annotated neighbour. chapter 6 reports three successive re-ranker versions (v1 per-aspect, v2 per-category, v3 full feature set) and their marginal gains over the score-fusion baseline.

eggNOG-mapper [10] deserves particular mention as a widely deployed homology-transfer system that combines DIAMOND alignment against clusters of orthologous groups (COGs) with GO term propagation. eggNOG-mapper's output is used as a reference benchmark in the CAFA 6 evaluation because it represents state-of-the-art homology transfer without pLM embeddings. PROTEA integrates an eggNOG-mapper annotation source plugin, allowing its output to serve as an additional evidence channel in the ensemble rather than as a competing system.

TALE [6] and comparable fine-tuned models implicitly perform a form of ensemble by combining PLM backbone representations with task-specific classifiers. The distinction between PROTEA's approach and fine-tuned classifiers is architectural: PROTEA does not modify the PLM weights but instead treats the PLM as a fixed feature extractor, which allows the same reference index to be updated (by adding newly annotated proteins) without retraining the backbone. This decoupling is central to the versioned-pipeline design described in chapter 4.

The re-ranking step in CAFA 6 top teams. Post-retrieval re-ranking emerged as a differentiating factor among CAFA 6 PLM-KNN systems. Systems that applied a learned re-ranker on top of initial KNN scores consistently improved over simple cosine-similarity-weighted aggregation. The feature space used by these re-rankers is broadly consistent across independent implementations: (i) cosine distance to each neighbour, (ii) the information content (specificity) of the candidate GO term under the Resnik measure, (iii) alignment-derived evidence where available (DIAMOND bit-score or E-value), (iv) taxonomic distance between query and annotated reference protein, and (v) GO-term embedding features (anc2vec or similar) that encode the ancestor structure of the candidate term. PROTEA's LightGBM re-ranker uses this feature set in three progressively richer configurations (v1, v2, v3), with ablation results reported in chapter 6. The offline training procedure and dataset construction are described in chapter 5.

3.7 Multiple Instance Learning and Interpretable Prediction

A growing body of work addresses interpretability in protein function prediction, motivated by the desire to understand not only whether a protein has a given function but which regions of the sequence are responsible.

OPUS-GO [52] (Yang et al., 2024, bioRxiv) applies multiple instance learning (MIL) to protein function prediction. The key insight is that a protein sequence can be viewed as a "bag" of residue-level instances, and the function of the whole protein is a label on the bag. OPUS-GO uses an attention-based MIL aggregator over per-residue ESM-2 embeddings, learning which residues contribute most to each predicted GO term. The attention weights serve as a soft annotation of functional sites, producing interpretable predictions that localise GO evidence to specific sequence positions.

PSALM [41] (MLSB 2024) extends the MIL paradigm with a probabilistic formulation that models uncertainty over both residue-level membership and bag-level labels. Its evaluation on CAFA targets showed calibration improvements over deterministic classifiers, making PSALM particularly relevant for downstream applications that require reliable confidence estimates (such as PROTEA's re-ranker, which consumes confidence scores as features).

Both OPUS-GO and PSALM are methodologically distinct from PROTEA's current pipeline (which performs kNN retrieval at the whole-sequence level using mean-pooled embeddings) but are directly relevant to PROTEA's F-MIL roadmap (documented in ADR D24), which plans to replace mean-pooled query representations with attention-weighted residue pooling informed by functional motifs. The F-MIL pilot would allow PROTEA to produce per-residue functional site predictions as a joint output alongside GO annotations, closing the interpretability gap relative to DeepFRI and OPUS-GO.

3.8 Ontology-Aware Representations

The Gene Ontology is a structured DAG, and several recent methods exploit that structure to inject ontology-aware signal into the prediction pipeline, rather than treating each GO term as an independent label.

anc2vec [13] learns a distributed representation of each GO term from its ancestor multiset under the *is-a* and *part-of* relations. Two terms with overlapping lineage have similar anc2vec vectors, providing a feature with which a downstream classifier can penalise predictions that contradict the true-path rule. PROTEA uses anc2vec features as an input to its LightGBM re-ranker, contributing the information-theoretic relationships between predicted and known terms as a structured feature.

GeOKG [25] extends this idea by embedding the GO graph in a multi-curvature space combining hyperbolic and Euclidean geometry, motivated by the observation that the GO DAG has both tree-like and grid-like substructure that no single-geometry embedding captures well. GeOKG-style embeddings are evaluated as a possible replacement for anc2vec in chapter 6.

Graph neural networks over the GO DAG have been investigated for end-to-end prediction [32]. They are an attractive direction for future work but lie outside the scope of the canonical PROTEA pipeline; chapter 7 discusses them in the context of the PROTEA-DL pilot.

3.9 Benchmarks and Evaluation Standards

3.9.1 CAFA: History and Methodology

The Critical Assessment of Protein Function Annotation (CAFA) [26] series is the field's longest-running community benchmark. CAFA 1 (2011, published 2013) [42] established the core protocol: participants receive query sequences before a submission deadline; ground-truth annotations are collected from experimental databases in the months following the deadline; submitted predictions are evaluated against this prospective ground truth. The approach avoids the circularity inherent in benchmarks that train and test on the same release of a database.

The primary metric is $F_{\max}$: the maximum F_1 score across all prediction confidence thresholds, computed separately for each GO aspect. AUPR (area under the precision-recall curve) and $S_{\min}$ (semantic distance) are complementary measures. PROTEA's evaluation in chapter 6 follows CAFA conventions.

CAFA 2 (2013, published 2016) [26] expanded the target set and introduced the $S_{\min}$ semantic distance metric alongside $F_{\max}$. The winning team used BLAST-based homology transfer with phylogenetic weighting, demonstrating that simple methods still dominated when applied carefully.

CAFA 3 (2017, published 2019) [55] was the largest edition to date, with over 100 participating teams. It highlighted the difficulty of the MF (Molecular Function) aspect and the value of combining alignment with learned features, prefiguring the embedding wave that followed.

CAFA 4 (2020) ran during the COVID-19 disruption and was not fully published as a community paper; results were released on Kaggle. Several teams incorporated ESM-1b embeddings for the first time, and kNN-based methods began to appear prominently.

CAFA 5 (2022, Kaggle competition format) [8] attracted over 1500 teams from the machine learning community, reflecting the broader interest in protein machine learning. The top submissions combined PLM embeddings with graph neural networks over the GO DAG and interaction networks. The scale of participation surfaced important questions about data leakage; several top-performing systems achieved high $F_{\max}$ by exploiting sequences that had been annotated in intermediate UniProt releases between target release and evaluation, rather than genuinely predicting novel functions.

An analysis of the CAFA 5 leaderboard reveals that the dominant winning pattern was kNN transfer from a PLM embedding space: the plurality of top-20 submissions used ESM-2 or ProtT5 embeddings with Swiss-Prot or GOA as the reference corpus, aggregating annotation scores over k nearest neighbours by cosine similarity. Graph-based models that encoded GO DAG topology provided incremental gains over flat KNN on the Molecular Function aspect, where the ontology's depth is most pronounced. Interaction network features (from STRING) contributed most on Biological Process, where pathway-level evidence is available, but were largely uninformative for Cellular Component and for orphan proteins lacking interactome coverage.

CAFA 6 (2024) [9] introduced stricter temporal splits and a leakage-controlled evaluation protocol, addressing the criticism that prior editions allowed training on sequences that overlapped with evaluation targets via intermediate databases. Among the CAFA 6 participants, the PLM-KNN family again dominated the top rankings, but the re-ranking step proved more important than in CAFA 5: systems that learned a secondary model over the initial KNN scores (using alignment bit-scores, taxonomy features, and GO-term information content as covariates) improved consistently over unweighted KNN transfer. This observation is the direct empirical motivation for PROTEA's LightGBM re-ranker. The PROTEA team participated in CAFA 6, achieving a second-place team result in the combined GO track; the present author was the technical lead responsible for the embedding pipeline, the FAISS index, and the re-ranking framework. The leakage question is addressed directly in PROTEA's evaluation protocol (chapter 6) through a temporal snapshot split analogous to the CAFA 6 protocol.

3.9.2 Recurring High-Rank Systems and Their Strategies

Several systems have placed consistently in the top tier across multiple CAFA editions and merit explicit discussion as reference points against which PROTEA's design choices are calibrated.

NetGO [53] and its CAFA 5 successor **NetGO 3** [54] represent the STRING-augmented PLM approach. Their architecture fuses ESM-2 sequence embeddings with protein-protein interaction context from the STRING database, producing per-aspect classifiers trained on GOA-derived labels. The multi-modal fusion consistently outperforms pure sequence methods on Biological Process but requires species-level interaction data that is unavailable for metagenomics targets.

TransFew [49] is a few-shot transfer method that treats each GO term as a class and computes query-to-support-set similarity in ESM-2 embedding space, weighting support proteins by their annotation reliability scores. The few-shot framing is well-suited to leaf-level terms where training examples number in the tens; it degrades gracefully compared to classifiers that require large positive sets.

FunBind [18] combines structure-based binding site matching (using AlphaFold-predicted residue contacts) with sequence-level PLM embeddings in a late-fusion architecture. Its binding-site component gives it a distinctive advantage on Molecular Function terms related to enzyme activity and ligand binding, where local structural context is more informative than global sequence identity.

GoCurator [21] is a more direct antecedent to PROTEA's design: it applies KNN retrieval in Swiss-Prot embedding space, then filters candidate annotations through a cascade of evidence-consistency checks (GO axiom compliance, evidence code weighting, taxon scope validation). The filtering step reduces false positives at the cost of recall on novel functions, a trade-off that PROTEA addresses differently by learning the filtering weights with LightGBM rather than applying fixed heuristics.

The convergence of these distinct architectures onto the PLM-KNN retrieval step as a shared foundation confirms that embedding-based nearest-neighbour transfer is now the load-bearing component of the state of the art. The principal axes of differentiation are: (a) the choice of PLM backbone, (b) the reference corpus and its temporal versioning, (c) the aggregation and re-ranking strategy, and (d) the integration of auxiliary evidence channels (structure, interaction networks, taxonomy). PROTEA occupies a specific position on each axis, documented in section 3.11.

3.9.3 The Leakage Question and NK/LK/PK Splits

Leakage in function prediction benchmarks arises when a protein in the evaluation set was already annotated (directly or via a known homologue) at the time the training data were collected. A method that memorises this annotation at training time will appear to "predict" it at evaluation time, inflating performance metrics. This problem is particularly acute for methods that use the full UniProt database (which is updated continuously) as a reference.

The **NK/LK/PK** (No Knowledge / Limited Knowledge / Prior Knowledge) split [42] addresses leakage by stratifying evaluation targets according to how much was known about them at submission time: **NK** targets had no annotation in any database at submission time (the hardest case), **LK** targets had partial annotation in non-GO sources, and **PK** targets were annotated in some GO aspects but not others. CAFA's official rankings report results across all three strata.

PROTEA's evaluation in chapter 6 adopts a temporal snapshot approach: two time-stamped GOA (Gene Ontology Annotation) database dumps are compared, and only proteins that received *new* experimental annotations between the two snapshots are included as evaluation targets. This strictly excludes any protein visible in the training-time reference database, providing a leakage-free evaluation that mirrors the CAFA protocol without requiring a community submission cycle.

3.9.4 Live Continuous Benchmarks: LAFA

The CAFA cycle releases a frozen evaluation snapshot every two to three years. A complementary movement towards always-on, container-based benchmarks has emerged with **LAFA** [40] (Live Assessment of Functional Annotation, `functionbench.net`). Participants submit a Docker container that consumes a FASTA file and emits predictions in a standardised format; the benchmark host runs the container against a periodically updated held-out set and publishes results continuously. LAFA lowers the barrier to public evaluation: any laboratory with a working method can submit without joining a formal challenge cycle. Three pre-built PROTEA configurations are submitted to LAFA in chapter 7.

3.10 Annotation Platforms and Pipelines

Several platforms have been developed to operationalise function prediction at genome scale.

Argot2 [17] combines BLAST and HMMER hits with a graph propagation scheme over the GO DAG. It produces confidence scores by integrating multiple lines of evidence, anticipating the multi-source ensemble strategy used in PROTEA.

PANNZER2 [48] is a web server that combines BLAST-based nearest-neighbour transfer with a SANS scoring model. It provides an interactive interface and supports batch submission, serving as a practical baseline for annotation quality in the twilight zone.

eggNOG-mapper [10] maps sequences against clusters of orthologous groups (COGs) and transfers functional annotations from the eggNOG database. It is widely used in comparative genomics pipelines and is a top-performing homology method in CAFA evaluations.

InterProScan (the command-line interface to InterPro) is the most widely deployed domain-matching pipeline in genome annotation workflows. Its output is used as a complementary evidence layer in several CAFA top-10 submissions and in PROTEA's domain-annotation source plugin.

None of these platforms provides a reproducible, versioned annotation pipeline backed by a relational data model that records which ontology version, reference annotation set, and embedding configuration produced each prediction. The lack of versioning is a systemic gap: two runs of the same platform six months apart may return different annotations because the underlying database was updated, with no way to reconstruct the earlier result from the platform's own logs. PROTEA addresses this gap directly through its contracts-first architecture (chapter 4), where every annotation artefact records a `schema_sha`, `goa_snapshot_date`, and `embedding_model_tag` in the relational schema. To the author's knowledge, no comparable bioinformatics annotation platform exposes this level of provenance as a first-class, schema-enforced property.

3.11 PROTEA's Position in the Landscape

Having surveyed the field, three positioning axes deserve explicit statement.

Platform versus method. PROTEA is simultaneously a *method* (the PLM-KNN+LightGBM re-ranker family that participated in CAFA 6) and a *platform* (the versioned, distributed pipeline that operationalises any combination of backends, sources, and inference strategies through a plugin contract layer). The platform contribution is independent of the method contribution: replacing the PLM-KNN backend with a fine-tuned classifier would still benefit from the platform's reproducibility infrastructure. This separation is novel in the bioinformatics annotation space, where platforms are typically tightly coupled to a single inference approach.

Leakage-free temporal evaluation. PROTEA's evaluation protocol, described fully in chapter 6, is one of few in the literature that systematically enforces temporal separation between reference data and evaluation targets. The NK/LK/PK splits from CAFA are replicated internally using GOA snapshot pairs, allowing comparable leakage-controlled reporting without a community benchmark submission. The quantitative contribution of the re-ranker over the baseline is reported exclusively on this leakage-free evaluation set.

Contracts-first architecture. The protea-contracts package defines abstract base classes and Pydantic payload schemas for every operation the pipeline supports. Annotation sources, embedding backends, and runner plugins implement these contracts; the core pipeline accepts any conforming implementation. This design pattern, borrowed from distributed systems engineering, is to the author's knowledge not present in any other published protein annotation pipeline. Its operational implications (zero-downtime backend swaps, schema-validated artefacts, testable plugin isolation) are discussed in chapter 4.

Table 3.1 summarises the methods discussed above along four axes relevant to this thesis.

PROTEA occupies a niche that none of the surveyed systems fills: an end-to-end distributed pipeline that combines pLM embeddings with optional alignment and taxonomy features, records all versioning information in a relational schema, exposes the full pipeline through a REST API and web interface, and supports runtime-swappable backends and sources through a typed contract layer.

Table 3.1: Comparison of protein function prediction approaches along four axes relevant to the PROTEA thesis: ability to annotate proteins below the twilight zone, practical scalability to large query sets, versioned and reproducible outputs, and open-source availability.

Method	**Family**	**Low-identity**	**Scalable**	**Versioned**	**Open source**
BLAST transfer	Alignment	×	✓	×	✓
PSI-BLAST	Alignment	∼	∼	×	✓
HMMER / Pfam	Profile HMM	✓	✓	×	✓
InterProScan	Profile HMM	✓	✓	×	✓
DeepGOPlus	Sequence deep	✓	✓	×	✓
DeepFRI	Structure deep	✓	∼	×	✓
TALE	PLM fine-tuned	✓	✓	×	✓
NetGO 3	PLM + network	✓	∼	×	✓
PANNZER2	Alignment KNN	×	✓	×	✓
eggNOG-mapper	Orthology	∼	✓	×	✓
ESM-2 KNN transfer	PLM KNN	✓	∼	×	✓
OPUS-GO	MIL PLM	✓	✓	×	✓
PROTEA	PLM KNN + rerank	✓	✓	✓	✓

Chapter 4

System Design

This chapter describes the architecture of PROTEA at the level of design decisions. Implementation details (specific libraries, migration tooling, frontend components) are deferred to chapter 5.

4.1 Requirements and Design Goals

The design of PROTEA is governed by five requirements derived from the limitations identified in chapter 3:

R1: Reproducibility. A prediction produced today must be exactly reproducible in the future. This requires recording the ontology version, reference annotation set, and embedding model configuration used for every prediction run.

R2: Scalability. The system must handle reference sets of hundreds of thousands of proteins and query sets of thousands without holding all data in memory simultaneously.

R3: Separation of concerns. Domain logic (what to compute), execution flow (how jobs are dispatched and tracked), and infrastructure (database, message queue) must be independently replaceable.

R4: Observability. Every job must produce a structured audit trail so that failures can be diagnosed without replaying the computation.

R5: Accessibility. Researchers without machine-learning infrastructure expertise must be able to submit sequences and retrieve predictions through a web interface or a REST API.

4.2 High-Level Architecture

PROTEA decomposes into four horizontal layers, illustrated in fig. 4.1:

Presentation layer. A Next.js single-page application exposes the user-facing workflow: uploading FASTA files, triggering pipeline jobs, monitoring progress, and downloading results.

API layer. A FastAPI application provides a REST interface. All state mutations flow through this layer; it writes job requests to PostgreSQL and publishes messages to RabbitMQ.

Worker layer. A set of Python worker processes consume messages from RabbitMQ queues. Workers are stateless with respect to domain logic: they resolve and execute registered operations and update job state in the database.

Data layer. PostgreSQL (extended with the pgvector extension for vector storage) is the single source of truth. RabbitMQ is used exclusively for task dispatch; it carries no persistent state.

4.3 The Operation Abstraction

The central design principle of PROTEA is the *Operation protocol*: every unit of domain logic is a class that implements exactly two members:

- `name: str`, a unique string identifier used to route messages.
- `execute(session, payload, *, emit) -> OperationResult`, the computation itself.

Operations receive a SQLAlchemy session and a validated Pydantic payload; they report progress through an `emit` callback that writes `JobEvent` rows to the database. Operations do not manage their own sessions, do not publish messages, and do not know about the queue infrastructure. This strict interface makes them independently testable and trivially substitutable.

The `OperationRegistry` maps string names to instances at startup. Workers resolve the correct operation by name when they receive a message, enabling new operations to be added without modifying any worker code.

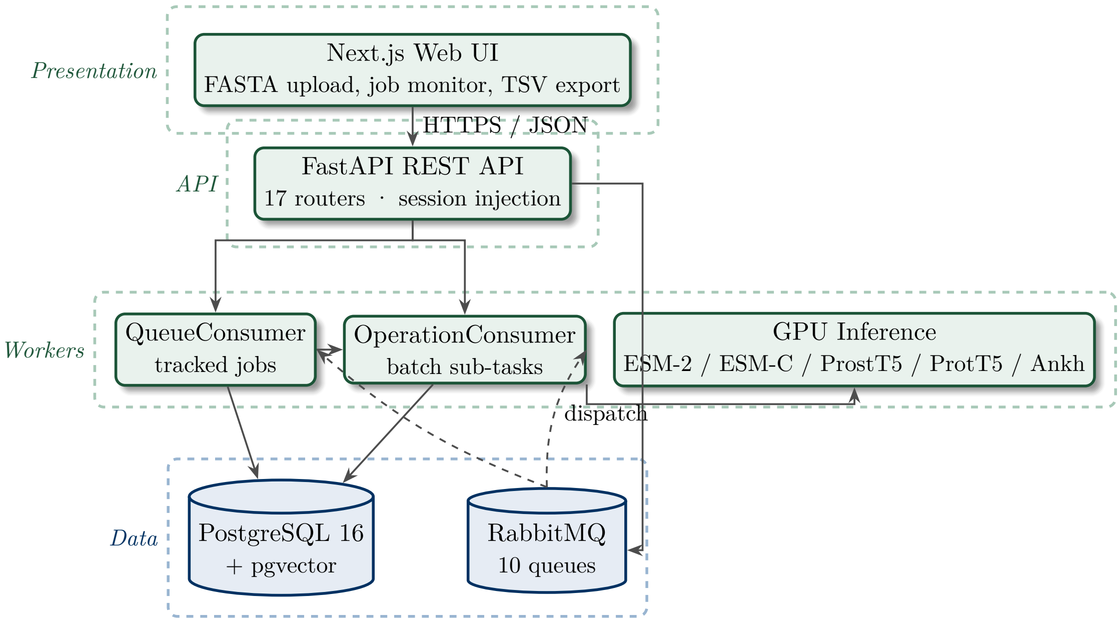

Figure 4.1: High-level architecture of PROTEA. The four horizontal layers (presentation, API, workers, data) communicate exclusively through well-defined interfaces: HTTPS/JSON between the browser and the API, RabbitMQ for task dispatch, and PostgreSQL as the single source of truth. GPU inference is isolated in dedicated workers so that compute capacity can be scaled independently of API throughput. Domain logic is factored across four plugin packages discovered at startup via Python entry points: `protea-method` (LAFA inference layer), `protea-backends` (protein language model backends: ESM-2, ESM-C, ProstT5, ProtT5, Ankh), `protea-runners` (experiment runners including LightGBM), and `protea-sources` (annotation sources: GOA, QuickGO, UniProt, InterPro). The offline reranker training laboratory (`protea-reranker-lab`) is not part of this plugin graph; it consumes frozen datasets published by PROTEA and feeds trained boosters back through the `/reranker-models` import endpoint.

4.4 Plugin Contracts and the Contracts Package

Operations describe the platform's internal vocabulary, but PROTEA also has to evolve along an orthogonal axis: a new protein language model becomes available, a new annotation source needs to be ingested, a different training algorithm warrants evaluation. If every such addition required edits to the core platform repository, the requirement R3 (separation of concerns) of section 4.1 would be undermined: domain logic and extension points would tangle, third parties could not contribute without forking the platform, and the cadence at which new capabilities can be shipped would be set by the slowest moving piece. The plugin layer answers that pressure.

4.4.1 The Contracts Package

A small Python package, `protea-contracts`, holds the abstract base classes and shared schemas that define the extension surface. The package is deliberately minimal: it depends only on `pydantic`, `numpy` and `pyarrow`, and is forbidden from importing `sqlalchemy`, `fastapi`, or anything from `protea-core`. Plugin repositories install `protea-contracts` without dragging the platform stack along; the inference path likewise consumes the package without paying for the worker, queue and ORM machinery. Three direct consequences follow.

First, plugin repositories can be developed, tested and released independently of `protea-core`. The five backend families shipped today (ESM-2 in three sizes, ESM-C, ProstT5, ProtT5, Ankh) live in a separate repository, are installable as pip install protea-backends[<extra>], and ship per-backend Poetry extras that pull only the heavy machine-learning libraries the chosen backend actually needs.

Second, the inference path ships as a standalone artefact, `protea-method`, without forcing downstream consumers into the platform's operational shape. The package exposes a single `pipeline.predict()` function that accepts already-loaded numpy arrays (query embeddings, reference embeddings, annotation maps) and returns structured prediction dicts. PROTEA's `predict_go_terms_batch` operation wires the DB-loaded inputs into this interface through a thin adapter; LAFA benchmark containers call the same function from bind-mounted parquet files. Any laboratory that wants to deploy a frozen model receives only the dozen megabytes of inference logic, without the platform's worker, queue, or ORM machinery as transitive dependencies.

Third, the contracts themselves become a SemVer-versioned public artefact. Adding a feature to the canonical schema bumps a minor; renaming an ABC method bumps a major. Consumers pin a known-good range and the platform team controls the breakage budget.

4.4.2 The Four Plugin Layers

Table 4.1 summarises the four extension points. Three of them are discovered through Python's `importlib.metadata.entry_points` mechanism: at startup the platform queries each named group, loads the plugin instances, and registers them in an in-process map analogous to the `OperationRegistry`. The fourth, the feature registry, is collected in-process from a fixed list of family modules within `protea-core` itself; it does not use entry points because the per-candidate features have to participate in a single, deterministic ordering for the schema fingerprint described below to be stable.

Table 4.1: The four plugin extension points of PROTEA.

Extension	**ABC**	**Discovery group**	**Examples**
Annotation sources	`AnnotationSource`	`protea.sources`	GOA, QuickGO, UniProt, InterPro
Embedding backends	`EmbeddingBackend`	`protea.backends`	ESM-2, ESM-C, ProstT5, ProtT5, Ankh
Experiment runners	`ExperimentRunner`	`protea.runners`	KNN, baseline, LightGBM
Per-candidate features	`FeatureRegistry`	in-process registry	alignment, taxonomy, anc2vec, emb-PCA

Each plugin is a Python module that subclasses the relevant ABC, sets a class attribute `name` matching the entry-point name, and exposes a module-level instance `plugin = MyPlugin()`. Heavy dependencies are imported lazily inside the methods that need them, never at module top. This last detail is load-bearing: `protea-core` startup queries the entry points and instantiates every discovered plugin, which would otherwise pay the import cost of `torch`, `transformers` and `esm` on every container start regardless of which backends are in use.

4.4.3 Discovery and Dispatch

Replacing a hard-coded `if/elif` chain with a registry lookup is a mechanical change with two non-obvious benefits. First, a missing or misspelt plugin name produces a single, readable error (`KeyError("unknown backend: ...")`) rather than the generic fall-through that the chain would have produced. Second, plugins added to the deployment after the binary was packaged become available without a rebuild: the platform image and the plugin packages are decoupled.

A sanity check guards the boundary: the `name` attribute on the loaded plugin instance must equal the entry-point name as declared in the plugin repository's `pyproject.toml`. A mismatch raises at startup rather than producing silent misrouting later. This is the kind of error that becomes invisible if it goes unchecked, since the misrouted plugin still implements the ABC correctly; the platform would happily call it, just under the wrong dispatch key.

4.4.4 Schema Drift and the Canonical Fingerprint

The most operationally consequential schema in `protea-contracts` is the canonical feature list of the re-ranker. Each candidate annotation in a `GOPrediction` row carries a fixed sequence of features (embedding distance, alignment statistics, taxonomic distance, ontology ancestry vectors, per-PLM PCA projections, annotation metadata). The ordering and the names matter: a LightGBM booster trained against feature `[a, b, c]` will silently apply learned weights to the wrong columns if it is later loaded against feature `[a, c, b]`. The output looks plausible, the metrics drift downward by a margin that is indistinguishable from ordinary model variance, and the bug remains invisible until something independent forces an audit.

To prevent that, the package exports a function `compute_schema_sha(features)` that reduces a list of feature names to a stable digest. The re-ranker registry stores the digest as `feature_schema_sha` on every `RerankerModel` row. At inference time, the live pipeline computes the digest from the active registry and refuses to apply a booster whose stored digest does not match. Adding a feature to the canonical list deliberately changes the digest: every booster trained against the old list becomes ineligible and must be retrained, which is the correct behaviour because the new feature has shifted the column space the booster understood.

The motivation for promoting this helper out of `protea-core` and into `protea-contracts` was not theoretical. During the 2026-05-01 evaluation campaign, an experimental run that should have been reproducible deviated by approximately three Fmax points across multiple ontology aspects. The eventual diagnosis traced the discrepancy to two parallel definitions of `compute_schema_sha`: one in the offline training repository (the *lab*), one inside `protea-core`. Both functions were correct in isolation, but a refactor to one had not propagated to the other, and the resulting digests differed for the same input feature list. Boosters registered through one code path were silently being scored under the other.

The fix combined three actions. First, the canonical implementation landed in `protea-contracts`, with a golden test pinning its output to a known digest. Second, the lab and `protea-core` were both amended to import the helper from the contracts package; their previously shadowed copies were deleted. Third, an Alembic migration introduced a parallel column `feature_schema_sha_v2` on the `RerankerModel` and `Dataset` tables, populated by a backfill script that recomputed each row's digest using the canonical helper. The legacy column is retained until the audit window closes; new registrations write only the v2 column. The episode is recorded in ADR D10.

4.4.5 Why a Separate Package, Not Just a Module

The contracts could in principle have lived inside `protea-core` as a designated public sub-module. Three considerations argued against that. The first is the inference shipping question above: the offline trainer and the LAFA submission containers consume contracts but cannot afford the platform stack as a transitive dependency. The second is the SemVer envelope: a sub-module's version moves with the platform, but the contracts evolve at a slower cadence and downstream consumers benefit from being able to pin them independently. The third is the schema drift incident itself, which would have been impossible if there had been a single canonical location for `compute_schema_sha` from the start. The discipline of packaging is the discipline of refusing to let two implementations drift apart in the first place.

4.5 Job Lifecycle and Worker Patterns

PROTEA distinguishes two worker patterns that correspond to different observability requirements:

4.5.1 QueueConsumer (tracked jobs)

Long-running, user-visible jobs (inserting proteins, loading ontologies, computing embeddings, predicting GO terms) are submitted through the API as `Job` rows with a JSONB payload. The workflow is:

1. The API creates a `Job` row in state `QUEUED` and publishes the job UUID to the appropriate RabbitMQ queue.
2. A `QueueConsumer` worker receives the UUID, opens a *claim session*, transitions the job to `RUNNING`, and flushes the `started_at` timestamp. The claim session is committed and closed before execution begins.
3. A separate *execute session* resolves the operation and calls `execute()`. On success the job transitions to `SUCCEEDED`; on exception to `FAILED`, with the error code and message stored on the row.

The two-session design ensures that the claim is durable even if the execute session rolls back. `JobEvent` rows, written via `emit()`, form an append-only audit log that records progress milestones, error context, and structured diagnostic fields.

4.5.2 OperationConsumer (fire-and-forget sub-tasks)

GPU-intensive batch tasks (running the embedding model on a group of sequences, or computing kNN predictions for a batch) do not create child `Job` rows. Instead, the coordinator operation publishes *operation messages* (containing the payload directly, not a UUID) to dedicated queues. `OperationConsumer` workers execute these messages and report progress via an atomic increment on the parent job's counter, avoiding write contention from hundreds of concurrent batches.

Figure 4.2 illustrates the two patterns and the queue topology.

4.5.3 Parent-Child Job Hierarchy

Coordinator operations (`compute_embeddings`, `predict_go_terms`) split work across many parallel workers using a parent-child pattern. The coordinator returns `OperationResult(deferred=True)`, which signals `BaseWorker` to *not* transition the parent job to `SUCCEEDED` immediately. Instead, the parent remains in state `RUNNING` while batch workers process their messages independently.

The `Job` model carries two progress fields for this purpose: `progress_total` (set by the coordinator to the number of batches dispatched) and `progress_current` (incremented atomically by each write worker). When the last write worker detects `progress_current` = `progress_total`, it closes the parent job as `SUCCEEDED`. This atomic counter design avoids write contention from hundreds of concurrent batches updating the same row.

4.5.4 RetryLaterError

When a shared resource is unavailable (for instance, the GPU is already occupied by a running embedding job), an operation raises `RetryLaterError` instead of failing permanently. `BaseWorker` catches this exception and:

1. Resets the job status back to `QUEUED`.
2. Writes a `job.retry_later` event recording the reason and the requested delay.
3. Re-publishes the job UUID to its queue after the specified delay (typically 60 seconds).

The embedding coordinator queue (`protea.embeddings`) is serialised precisely because of this mechanism: at most one embedding coordinator runs at a time, with subsequent requests queued and retried automatically.

4.5.5 Cancellation

`POST /jobs/{id}/cancel` transitions a `QUEUED` or `RUNNING` job to `CANCELLED`. If the job is already in a terminal state (`SUCCEEDED`, `FAILED`, or `CANCELLED`) the request is a no-op. Any queued child jobs are also cancelled atomically in the same transaction.

Cancellation of a `RUNNING` job is a *soft cancel*: the database row is marked `CANCELLED` immediately, but the worker process is not interrupted. The worker will still complete its operation and attempt a final status write; however, because `CANCELLED` is already committed, the frontend reflects the cancelled state regardless of how the worker exits.

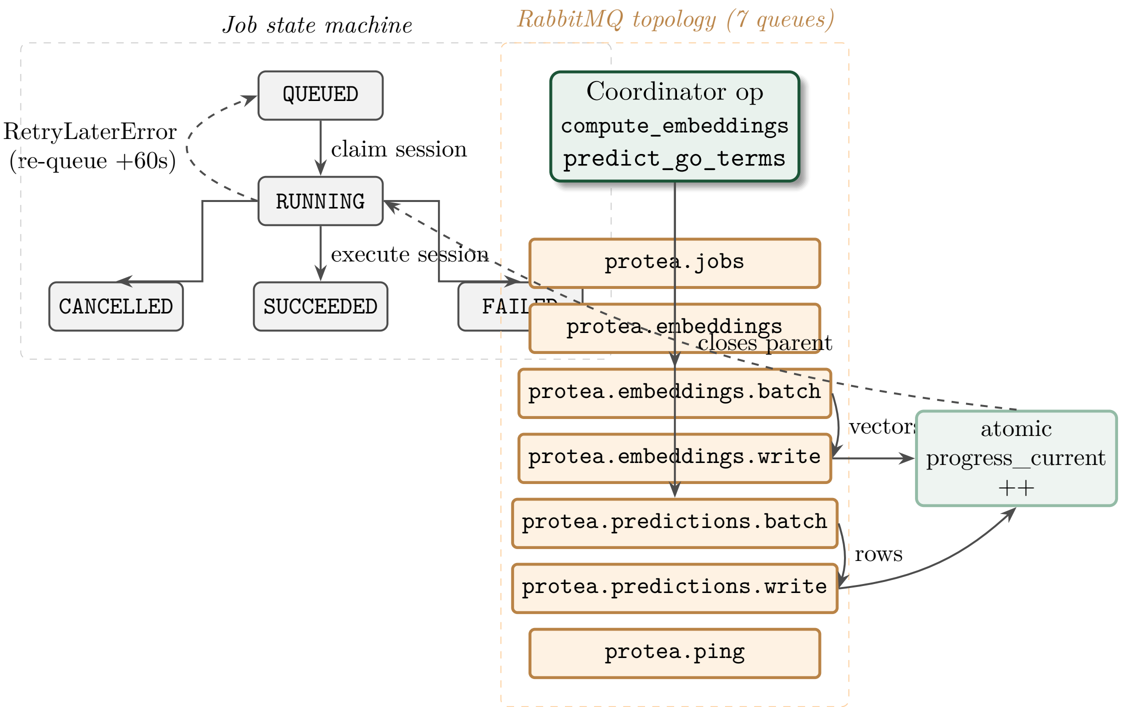

Figure 4.2: Job lifecycle and queue routing in PROTEA. The state machine on the left is enforced by `BaseWorker` through two independent SQLAlchemy sessions: a short *claim session* that commits the `QUEUED` → `RUNNING` transition, and a longer *execute session* that runs the operation and commits the terminal state. `RetryLaterError` resets the job to `QUEUED` and re-publishes it after a delay (typically 60 s) so that GPU contention never produces hard failures. The right column shows the ten RabbitMQ queues; coordinator operations fan out work into the `*.batch` → `*.write` chains, where every write worker atomically increments `progress_current` on the parent job. The last write closes the parent as `SUCCEEDED`, avoiding write contention from hundreds of concurrent batches.

4.6 Queue Topology

PROTEA routes messages across ten RabbitMQ queues, each with a distinct semantic purpose:

`protea.ping` Smoke-test queue. Used to verify end-to-end connectivity.

`protea.jobs` General-purpose job queue. Handles protein insertion, metadata fetch, ontology loading, annotation loading, and the `generate_evaluation_set` operation.

`protea.training` Serialised export queue. Runs `export_research_dataset`, which performs the GPU/RAM-intensive KNN pass, feature engineering, and artefact-store upload that produces the frozen `train.parquet` / `eval.parquet` dataset consumed by the offline reranker laboratory. Re-ranker training no longer runs inside PROTEA; the training function has been absorbed into `protea-runners.lightgbm` as a plugin (ADR D1, D14).

`protea.embeddings` Serialised embedding coordinator queue. At most one coordinator runs at a time; if the GPU is busy the coordinator raises `RetryLaterError` and is re-queued with a 60-second delay.

`protea.embeddings.batch` GPU inference queue. Each message carries a batch of sequences; `OperationConsumer` workers run the forward pass of the protein language model and publish results to the write queue.

`protea.embeddings.write` Bulk insert queue. Workers receive computed embedding vectors and write them to the `SequenceEmbedding` table via pgvector.

`protea.predictions` Serialised prediction coordinator queue. The `predict_go_terms` coordinator partitions the query set into batches and fans them out to `protea.predictions.batch`.

`protea.predictions.batch` kNN prediction queue. Each message carries a batch of query embeddings; `OperationConsumer` workers search the reference index and transfer GO annotations.

`protea.predictions.write` Bulk prediction insert queue. Workers write `GOPrediction` rows for a batch of query-reference pairs.

`protea.evaluations` CAFA evaluation queue. Runs `run_cafa_evaluation` (Fmax, AuPRC, coverage across NK/LK/PK subsets). Serialised and isolated so that long evaluation runs do not block the general jobs queue.

This topology separates GPU-bound inference, I/O-bound database writes, and CPU-bound coordination into independent queues that can be scaled independently (e.g. `manage.sh scale protea.predictions.batch 4`).

4.7 Data Model

The relational schema is organised into five logical groups:

4.7.1 Sequence and Protein

`Sequence` stores deduplicated amino acid strings, keyed by MD5 hash. `Protein` stores one row per UniProt accession; multiple accessions (canonical and isoforms) may reference the same `Sequence`. This deduplication prevents redundant embedding computation.

4.7.2 Ontology and Annotations

`OntologySnapshot` records one complete GO release (OBO format), versioned by the `obo_version` string found in the OBO file header. `GOTerm` and `GOTermRelationship` store the DAG within a snapshot.

`AnnotationSet` groups a batch of `ProteinGOAnnotation` rows by source (`goa` or `quickgo`) and links them to an `OntologySnapshot`. This design enables side-by-side comparison of annotation sets from different sources or dates.

4.7.3 Embeddings

`EmbeddingConfig` is an immutable recipe: it records the model identifier, chunking strategy, pooling method, and any other parameters that affect the embedding geometry. Its UUID primary key is stable; changing any parameter creates a new configuration.

`SequenceEmbedding` stores one pgvector `VECTOR` per (`sequence`, `config`, `chunk`) triple. Chunking support allows sequences that exceed the model's context window to be split into overlapping segments.

4.7.4 Predictions

`PredictionSet` is the result container, linking a query set, an embedding configuration, an annotation set, and an ontology snapshot. `GOPrediction` stores one row per (query protein, GO term) pair, including the cosine distance from the nearest reference neighbour and the optional feature-engineering columns described in section 5.5.

4.7.5 Reranker Models

`RerankerModel` records a trained LightGBM booster promoted from the research sandbox described in section 5.9. The row carries the artefact URI resolved by PROTEA's storage abstraction (`file:///...` for local filesystem or `s3://...` for MinIO), the `feature_schema_sha` fingerprint used to validate feature-set alignment at inference time, the originating embedding configuration and ontology snapshot, and the full experiment specification YAML used to reproduce the run. The inline `model_data` column is retained only as a nullable compatibility slot for rows produced by the pre-abstraction pipeline; new registrations never populate it.

4.7.6 Query Sets

`QuerySet` represents a user-uploaded FASTA dataset submitted for custom prediction queries. Each `QuerySetEntry` row preserves the original accession header from the FASTA file and links to the deduplicated `Sequence` row, reusing existing sequences when the amino-acid string is already present in the database. Query sets are created via `POST /query-sets` and can be referenced in subsequent embedding computation and prediction jobs.

4.7.7 Jobs

`Job` implements a state machine with states `QUEUED`, `RUNNING`, `SUCCEEDED`, `FAILED`, and `CANCELLED`. `JobEvent` is an append-only log of timestamped, structured events emitted during execution. Both tables use JSONB columns for extensible payloads and metadata.

4.8 Embedding Pipeline

The embedding pipeline runs in three phases coordinated through the queue topology described above:

1. **Coordinator** (`compute_embeddings`): queries the database for all sequences in the target set that do not yet have an embedding for the requested `EmbeddingConfig`; partitions them into fixed-size batches; publishes one operation message per batch to `protea.embeddings.batch`.

2. **Batch inference** (`compute_embeddings_batch`): resolves the `EmbeddingConfig.model_bac` key against the `protea.backends` plugin registry; calls `plugin.load_model()` and `plugin.embed_batch()` on the discovered backend instance. Each backend plugin (ESM-2, ESM-C, ProstT5, ProtT5, Ankh) owns its own torch/transformers imports and the device and dtype orchestration; the core operation is agnostic of the model family. The resulting float16 matrix is mean-pooled into one vector per chunk and published to `protea.embeddings.write`.

3. **Store** (`store_embeddings`): receives the computed vectors and performs a bulk upsert into `SequenceEmbedding`.

The coordinator is serialised (one at a time per the `protea.embeddings` queue) to prevent concurrent model loads from exhausting GPU memory. Batch and write workers can be scaled horizontally.

4.9 Prediction Pipeline

The prediction pipeline mirrors the structure of the embedding pipeline:

1. **Coordinator** (`predict_go_terms`): loads the reference embeddings (all proteins in the annotation set that have embeddings) into a process-level cache compressed to float16; partitions the query set into batches; publishes one operation message per batch to `protea.predictions.batch`.

2. **Batch prediction** (`predict_go_terms_batch`): delegates KNN search, GO annotation transfer, v6 feature enrichment, and optional reranking to `protea_method.pipeline.predict()`. The operation class is responsible for loading reference embeddings and annotations from the database, wiring them into the pure-function interface through a thin adapter, expanding predictions to ancestor GO terms, and publishing the resulting rows to `protea.predictions.write`.

3. **Store** (`store_predictions`): bulk-inserts `GOPrediction` rows.

The reference cache is held at the process level and is keyed by (embedding config, annotation set). It holds at most one entry to bound memory usage; a restart reloads from the database automatically.

4.9.1 Unified and Aspect-Separated KNN Modes

Two KNN modes are available, selected via the `aspect_separated` flag on the batch payload:

Unified KNN A single index is built across all reference embeddings regardless of GO aspect. One search per query yields the k globally nearest neighbours; their annotations are pooled across all three GO aspects.

Aspect-separated KNN Three independent indices are built, one per GO aspect (Biological Process, Molecular Function, Cellular Component), each restricted to references that carry at least one annotation in that aspect. The results from all three searches are merged. This mode guarantees per-aspect candidates even when a query's globally nearest neighbours happen to carry annotations in only one or two aspects, which is the dominant cause of the BPO recall ceiling on a unified index.

Both modes share the same adapter entry-point to `protea_method.pipeline.predict()` and the same downstream store path; switching between them changes only the argument passed to the orchestrator.

When a job payload includes a `reranker_model_id`, the batch worker performs one additional pass after the kNN stage: the booster referenced by the registered `RerankerModel` is loaded through a sha-keyed on-disk cache, the candidate rows are scored, and the resulting `reranker_score` values replace the kNN distance as the ranking key. The pass is skipped and a `reranker.schema_mismatch` event is emitted if the feature fingerprint computed from the live pipeline does not match the one registered with the booster; the job otherwise proceeds to the store stage unchanged. The promotion mechanics are described in section 5.9.

4.10 Artifact Store

Large produced blobs (trained LightGBM boosters, exported research datasets comprising `train.parquet`, `eval.parquet`, and `manifest.json`) are not stored in PostgreSQL. Placing binary model artefacts in a relational database would conflate the roles of the transactional store and the object store, complicate backup strategies, and make it impractical to share datasets with the offline training environment. Instead, PROTEA writes these blobs through an `ArtifactStore` protocol defined in `protea/infrastructure/storage/`.

The protocol exposes four methods (`put`, `get`, `url`, `exists`) and two concrete implementations:

LocalFsArtifactStore The default backend. Blobs are written to a configurable directory on the API host (by convention storage/artifacts/). URIs are file:///... paths. Suitable for single-node deployments and development.

MinioArtifactStore An S3-compatible object store, activated by setting storage.backend: minio in system.yaml and starting the compose storage profile. URIs are s3://<bucket>/<key>. Suitable for multi-node deployments and cloud environments.

Operation code is agnostic of which backend is active: it calls ArtifactStore.put() and receives back a URI string that is then stored in the corresponding ORM row (RerankerModel.artifact_uri or Dataset.artifact_uri). If MinIO is configured but unreachable at startup, the factory logs a warning and degrades to the local filesystem; a missing optional service never crashes the stack. This graceful degradation satisfies requirement R3 (separation of concerns): the storage backend is independently replaceable without modifying any operation logic.

4.11 REST API Design

The API follows REST conventions. Resources are grouped into seventeen routers spanning the full operational surface of the platform (ADR D4):

/proteins Insert proteins from UniProt accession lists or FASTA; fetch and cache UniProt metadata.

/annotations Load ontology snapshots and annotation sets from GOA or QuickGO sources.

/embeddings Manage embedding configurations, trigger computation, submit prediction jobs, and export results.

/query-sets Upload FASTA files and manage user query datasets.

/jobs Track job state and stream structured events.

/admin Administrative and maintenance tasks (schema health, stale-job reaping).

/scoring Weighted composite scoring of candidate GO predictions with configurable formulas.

/annotate High-level annotate endpoint; accepts a FASTA and returns structured GO predictions in a single synchronous call.

`/benchmark` Trigger and retrieve CAFA-style evaluation runs (Fmax, AuPRC, coverage) against registered evaluation sets.

`/datasets` Manage exported research datasets (the frozen `train.parquet` / `eval.parquet` artefacts consumed by the reranker lab).

`/reranker-models` Register promoted LightGBM boosters, inspect their `feature_schema_sha`, and set an active model for inference.

`/registry` Inspect registered plugins (backends, sources, runners) and available operations at runtime.

`/experiment-runs` Record and retrieve experiment provenance: parameters, findings, and tags attached to a prediction or evaluation job.

`/support` Diagnostic and support endpoints used by the frontend for connectivity checks and stack introspection.

`/showcase` Read-only endpoints for pre-computed demonstration predictions surfaced in the public-facing interface.

`/stack` Runtime stack inspection: worker counts, queue depths, and database connectivity status.

`/maintenance` Database maintenance tasks (vacuum, index rebuild, schema repair) exposed for operational use.

Session injection follows FastAPI's dependency system: `app.state.session_factory` is set at startup and injected into each request handler, keeping router code free of infrastructure imports. The strict `extra=forbid` configuration on all `ProteaPayload` subclasses means that a malformed request body fails at HTTP 422 before any database write occurs.

4.12 Internal Design Patterns

PROTEA's operations contain non-trivial coordination logic. Two recurring structural patterns keep that logic tractable as the codebase evolves.

4.12.1 Parameter Object

Long-running operations accumulate state that must be threaded through multiple private helpers. Passing each field as a separate argument pushes function signatures beyond a manageable width and makes refactoring expensive: any addition or removal of a field requires updating every call site.

The Parameter Object pattern addresses this by grouping related values into a frozen dataclass or `NamedTuple`. PROTEA uses this pattern systematically:

`AspectSeparatedKnnContext` Bundles the per-batch inputs for the aspect-separated KNN path (query embeddings, per-aspect reference pools, annotation-set and prediction-set identifiers, and the full batch payload). Every private helper in the batch operation receives this context rather than the same six arguments individually.

`_UnifiedPredictContext` Analogous bundle for the unified-KNN path: query embeddings, cached reference data, and the resolved set identifiers.

`KnnTransferContext` Groups the inputs for the KNN-transfer-and-label step inside the training-dump operation: annotation snapshot pair, embedding config, and KNN parameters.

`RetryPolicy` Bundles the six tunable knobs of the `with_retry` transient-error middleware (maximum attempts, base delay, maximum delay, jitter ratio, retry predicate, and initial delay override) into a single value that can be constructed once and passed to every call site without repeating defaults.

The pattern has an important secondary benefit: when an operation is split into a Method Object (see section 4.12.2), the existing context dataclass becomes the constructor argument for the new class without any further refactoring at the call sites.

4.12.2 Method Object at Sub-Cluster Granularity

The classic Replace-Method-with-Method-Object refactoring encapsulates one large method into a dedicated class whose constructor receives the method's arguments and whose fields replace local variables. PROTEA applies a refinement of this pattern: rather than one Method Object per *entry point*, a Method Object is introduced per cohesive *sub-cluster* of helpers that share mutable state.

The motivation is empirical. During the T2B.5 refactoring pass, static analysis confirmed that the four originally targeted `execute()` methods had already been reduced to well under the 60-LOC budget by earlier extraction rounds. The residual smell was not the size of any single method but the breadth of the parent class whose private helpers spanned unrelated concerns. The appropriate unit of encapsulation was therefore the cohesive sub-cluster rather than the entry point.

In the prediction batch operation this produced `_AspectSeparatedKnnRunner`, a class that owned the nine private methods implementing the aspect-separated KNN path and exposed a single `run()` entry point; the original `_run_aspect_separated_knn` became a one-line forwarder. When F2C.5b subsequently delegated this path entirely to `protea_method.pipeline.predict()`, the runner was dissolved because its responsibilities had moved to the library layer; only the thin orchestrator method and the adapter module remained. The sub-cluster pattern thus served its purpose as a transitional structure rather than a permanent one.

The design principle carried forward is: the correct granularity for a Method Object is the smallest set of methods that share mutable state and have no coupling to the surrounding class beyond that shared state. Applying the pattern at this granularity keeps classes navigable and avoids prematurely freezing the structure of code that is still evolving.

4.13 Design Decision Index

The architecture described in this chapter is the outcome of a series of design decisions recorded as Architectural Decision Records (ADRs) in the PROTEA repository. The following index maps the major design themes of this chapter to their authoritative ADRs; the full text of each record is available in `docs/source/adr/` in the PROTEA repository (Appendix B summarises the most consequential decisions).

Repository and plugin structure (D1, D14). ADR D1 establishes the seven-repository split (core, contracts, method, sources, backends, runners, cafaeval) and the entry-points discovery mechanism. D14 defers per-plugin repository granularity to post-defense (F9), keeping each plugin group in a single repo for now.

KNN without pgvector search (D1, referenced as ADR-001). pgvector is used for vector storage only; KNN search runs in Python via NumPy or FAISS. The constraint is a hard project rule (see chapter 5).

Two-session worker pattern (ADR-002). The claim session and execute session separation described in section 4.5.1 ensures that job claims survive execute-session rollbacks.

QueueConsumer versus OperationConsumer (ADR-003, D11). The two consumer patterns and their observability trade-offs are formalised in ADR-003. D11 adds the narrative model (`description`, `findings`, `tags`) to `Job` rows.

Dead-letter and retry strategy (ADR-004). `RetryLaterError` and the 60-second re-queue mechanism described in section 4.5.4 are specified here.

Thread-local RabbitMQ connections (ADR-005). Each worker thread holds its own channel; sharing a single connection across threads would require locking that the aio-pika layer does not provide.

Sequence deduplication by MD5 (ADR-006). The `Sequence`/`Protein` split described in section 4.7.1 prevents redundant embedding computation.

Contract-first lab integration (ADR-007). The narrow interface between PROTEA and the offline reranker laboratory (frozen parquet datasets, booster artefacts, and the shared `compute_schema_sha` helper) is specified in ADR-007. ADR D9, which originally proposed shipping the lab as a runtime dependency, has been superseded and declared obsolete: under Structure C (ADR D1) the LightGBM training function lives in `protea-runners.lightgbm` as a plugin, and the lab's only coupling to the platform is the dataset-publishing contract (Dataset row and artefact store URI).

cafaeval coverage fix (ADR-008). An upstream bug in the cafaeval library that affected PK coverage reporting is documented here; the fix was contributed upstream and backported.

Schema SHA v2 migration (D10). The parallel `schema_sha_v2` column migration described in section 4.4.4 is fully specified in D10, including the production rollout gate (staging verification before live migration).

Exportable research dataset location (D2). The `Dataset` ORM model and the frozen parquet artefact layout (`train.parquet`, `eval.parquet`, `manifest.json`) are specified here.

GOPrediction features in JSONB (D3). The extensible JSONB column on `GOPrediction` that carries per-candidate features without schema migrations is specified in D3.

API versioning (D4). The decision not to version the API path (no `/v1/`) and to use contract stability through strict payload validation instead is recorded in D4.

Frontend co-located in core (D5). The Next.js application lives inside the PROTEA monorepo under `apps/web/` rather than in a separate repository (D5, D8, D13).

Authentication scope (D6). Single-instance, researcher-facing deployment does not implement authentication at this stage; the decision and its deferral conditions are in D6.

Observability stack (D7). OpenTelemetry traces, Prometheus metrics, Grafana dashboards, and Loki log aggregation are specified in D7.

`protea-method` distribution (D15). The three distribution channels (PyPI, Docker Hub minimal image, companion engineering paper) for the standalone inference path are specified in D15.

Container runtime (D26). OCI multi-stage Dockerfiles as the source of truth; Apptainer `.sif` images generated as release artefacts for HPC environments.

HPC operation mode (D25). Mode B (stateless workers on HPC nodes connecting to cloud-hosted Postgres and RabbitMQ) is the primary HPC deployment target; mode C (airgapped `.sif` bundle) is deferred to post-defense.

Secrets management (D28). Credentials are never committed; the canonical pattern is environment variables injected at container start.

Release pipeline (D29). Per-repo SemVer driven by Conventional Commits and `release-please`; cross-repo integration test on tag.

Operational insights appendix (D30). Consequential incidents (schema SHA drift, anc2vec feature leakage, cafaeval coverage bug) are documented in Appendix B rather than buried in commit history.

T2B.5 Method Object reframe (D31). The refactor strategy for large batch operation classes (extracting cohesive sub-clusters as `_<Cluster>Runner` classes rather than wrapping already-budgeted `execute()` methods, see section 4.12.2) is formalised in D31.

Chapter 5

Implementation

This chapter describes the concrete technology choices and key implementation details behind the design presented in chapter 4.

5.1 Project Structure

The PROTEA codebase is organised into three top-level packages that mirror the architectural layers described in chapter 4:

`protea/api/` FastAPI application and seventeen routers spanning the full operational surface of the platform. The routers are described in section 4.11.

`protea/core/` Pure domain logic: the `Operation` protocol and `OperationRegistry` contracts, all registered operations, the kNN search backends, feature engineering utilities, and shared HTTP helpers.

`protea/infrastructure/` SQLAlchemy ORM models, RabbitMQ consumer and publisher, the session context manager, and the configuration loader.

Two additional directories complete the repository:

`apps/web/` The Next.js frontend application.

`scripts/` Operational tooling: `manage.sh` (process manager), `worker.py` (worker entry point that registers all operations), `init_db.py` (schema initialisation), and `run_one_job.py` (manual job runner for debugging).

This layout enforces the dependency direction: `api` and `workers` depend on `core` and `infrastructure`; `core` has no dependency on either of the other two.

5.2 Technology Stack

Table 5.1 summarises the main components of the PROTEA stack.

Table 5.1: PROTEA technology stack.

Component	Technology	Version
API framework	FastAPI	0.115+
ORM / migrations	SQLAlchemy 2.0 + Alembic	2.0 / 1.13
Database	PostgreSQL 16 + pgvector	16 / 0.7
Message broker	RabbitMQ + aio-pika	3.x / 9.x
Data validation	Pydantic v2	2.x
Protein LM inference	Hugging Face Transformers	4.x
Alignment	parasail-python (BLOSUM62)	1.x
Taxonomy	ete3 + NCBITaxa	3.x
ANN search	NumPy / FAISS	n/a
Frontend	Next.js 16 + Tailwind v4	16 / 4
Dependency mgmt.	Poetry	1.x

All Python dependencies are declared in `pyproject.toml` with pinned version ranges; `poetry.lock` guarantees reproducible installs across environments. The `dev` dependency group adds pytest, pytest-cov, and related tooling without polluting the production image.

5.3 Database Schema and Migrations

The database schema is managed with Alembic. Migration scripts are generated from SQLAlchemy ORM metadata via `alembic revision -autogenerate` and stored under `alembic/versions/`. The Alembic `env.py` reads the database URL from `protea/config/system.yaml` (or the `PROTEA_DB_URL` environment variable), ensuring consistency with the application configuration.

The pgvector extension is enabled at database initialisation time:

Listing 5.1: Enabling pgvector.

```sql
CREATE EXTENSION IF NOT EXISTS vector;
```

`SequenceEmbedding` columns are declared as `VECTOR(n)` where n is determined by the embedding model (e.g. 1280 for ESM-2 650M, 2560 for ESM-2 3B). Although pgvector supports index types for nearest-neighbour queries (HNSW, IVFFlat), PROTEA intentionally does not use them for kNN search: querying through pgvector at the scale of hundreds of thousands of vectors introduces unacceptable latency. Instead, reference embeddings are loaded into Python (NumPy or FAISS) at prediction time.

5.3.1 Session Management

All database access goes through `session_scope()`, a context manager that commits on normal exit and rolls back on exception:

Listing 5.2: Session context manager.

```
@contextmanager
def session_scope(factory):
    session = factory()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()
```

The two-session pattern in `BaseWorker` (claim session, execute session) relies on this primitive: the claim commits independently of the execute, so a crash during execution never rolls back the `RUNNING` state transition.

5.4 Operations

The `build_operation_registry()` factory in `protea/core/operation_catalog.py` registers the full set of operations at startup; both the API process and every worker call the same factory so the set is always in sync.

The user-facing operations are:

`ping` Smoke test. Returns immediately with a success result.

`insert_proteins` Paginates the UniProt REST API using cursor-based FASTA streaming. Sequences are deduplicated by MD5 hash before upsert; proteins are upserted by accession. Exponential backoff with jitter and `Retry-After` header handling are implemented in the shared `UniProtHttpMixin`.

`fetch_uniprot_metadata` Downloads TSV functional annotation data from UniProt and upserts `ProteinUniProtMetadata` rows keyed by canonical accession.

`load_ontology_snapshot` Downloads a GO OBO file and populates `OntologySnapshot`, `GOTerm`, and `GOTermRelationship` rows. The `obo_version` field carries a unique constraint so that re-importing the same release is idempotent.

`load_goa_annotations` Bulk-loads a GO Annotation File (GAF) file. Annotations are filtered against canonical accessions present in the database, avoiding orphaned foreign keys.

`load_quickgo_annotations` Streams GO annotations from the QuickGO bulk download API (paginated TSV). Supports optional ECO→GO evidence code mapping and per-page commits to bound transaction size.

`generate_evaluation_set` Materialises an `EvaluationSet` by snapshotting ground-truth annotations for a target ontology snapshot and accession set; consumed downstream by `run_cafa_evaluation`.

`run_cafa_evaluation` Runs the standalone `cafaeval` fork against a `PredictionSet` and persists `EvaluationResult` rows (Fmax, AuPRC, coverage, per-aspect breakdowns). Isolated on its own queue so long evaluations do not block the general jobs queue.

`compute_embeddings` Coordinator operation described in section 4.8.

`predict_go_terms` Coordinator operation described in section 4.9.

`export_research_dataset` Writes the frozen train/eval parquet dataset and its `manifest.json` used by the external research sandbox described in section 5.9. The operation runs the same KNN and feature-generation pipeline as the internal `training_dump_helpers` module, skips the LightGBM training stage, and publishes the resulting artefacts through the storage abstraction (section 4.7.5).

5.5 Feature Engineering

When a prediction job is submitted with `compute_alignments=true` or `compute_taxonomy=true`, the batch prediction operation augments each query-reference pair with additional numerical features.

5.5.1 Pairwise Alignment

For each (query, reference) pair, PROTEA computes global (NW) and local (SW) alignments using the `parasail` library with BLOSUM62 substitution scores and gap-open/gap-extend penalties of $-11/-1$. The following derived statistics are stored as columns on `GOPrediction`:

- `identity_nw/sw`: fraction of aligned positions with identical residues.
- `similarity_nw/sw`: fraction of aligned positions with positive BLOSUM62 score.
- `alignment_score_nw/sw`: raw alignment score.
- `gaps_pct_nw/sw`: fraction of the alignment that is gap.
- `alignment_length_nw/sw`: number of aligned columns.
- `length_query / length_ref`: sequence lengths.

5.5.2 Taxonomic Distance

For each (query, reference) pair where taxonomy IDs are available from UniProt metadata, the NCBI taxonomy tree is consulted via ete3 to compute:

- `taxonomic_lca`: the taxon ID of the LCA.
- `taxonomic_distance`: the total edge count between the two taxa through their LCA.
- `taxonomic_common_ancestors`: the number of shared ancestors.
- `taxonomic_relation`: a categorical label: *same_species*, *same_genus*, *same_family*, *same_order*, *same_class*, *same_phylum*, *same_kingdom*, or *distant*.

Taxonomy lookups are memoised with an LRU cache keyed by taxon ID pair to avoid redundant tree traversals across a batch.

5.6 Approximate Nearest-Neighbour Search

The kNN search backend is configurable per prediction job via the `search_backend` payload field. Two backends are supported:

`numpy` Computes exact cosine distances as a matrix product followed by L2 normalisation. This is $O(NQ)$ in both time and memory, but is acceptable for reference sets up to ∼100k proteins when the reference embeddings fit in RAM as float16 (approximately 250 MB for 100k × 1280 dimensions).

`faiss` Uses the FAISS library [27] with a configurable index type. `IVFFlat` partitions the vector space into Voronoi cells and restricts the search to the `nprobe` nearest cells, reducing query time from $O(N)$ to approximately $O(\sqrt{N})$ with negligible recall loss for typical parameter settings.

The pgvector `VECTOR` type is used solely for storage and retrieval; nearest-neighbour queries are never issued via SQL, as pgvector's index performance degrades at the scale of hundreds of thousands of vectors under concurrent load.

5.7 Prediction Export

Prediction results can be exported as a tab-separated file through the endpoint `GET /embeddings/prediction-sets/{id}/predictions.tsv`. The response uses `StreamingResponse` with `yield_per(1000)` to avoid loading the full result set into memory. The exported file contains 27 columns covering protein accessions, GO term, cosine distance, reference evidence code, alignment statistics, and taxonomy fields. Optional query-string filters allow the client to restrict the export by accession, GO aspect (`F`/`P`/`C`), or maximum distance threshold.

5.8 Process Management

PROTEA uses a shell script (`scripts/manage.sh`) to manage the set of long-running processes required by the stack. The script starts, stops, and reports the status of all process roles:

1. FastAPI server (`uvicorn`)
2. Worker: `protea.ping`
3. Worker: `protea.jobs`

4. Worker: `protea.training` (serialised dataset-export queue)
5. Worker: `protea.embeddings` (serialised coordinator)
6. $N\times$ Worker: `protea.embeddings.batch` (GPU inference)
7. $N\times$ Worker: `protea.embeddings.write`
8. Worker: `protea.predictions` (serialised coordinator)
9. $N\times$ Worker: `protea.predictions.batch` (kNN)
10. $N\times$ Worker: `protea.predictions.write`
11. Worker: `protea.evaluations` (CAFA evaluation queue)
12. Stale-job reaper (`reaper` queue, periodic heartbeat check)
13. Next.js frontend server

PID files and log files are written under `logs/`. The `manage.sh scale` subcommand adds extra batch workers to a queue without restarting the rest of the stack.

5.9 Reranker Promotion Pipeline

The learning-to-rank reranker component is developed in a companion repository, `protea-reranker-lab`, kept deliberately separate from PROTEA to protect the production image from the research dependency tree (LightGBM, Pandas, the training runtime). The two repositories exchange information through a narrow contract rather than direct imports, so that a regression in the research sandbox can never degrade serving behaviour.

The contract consists of three artefacts:

`manifest.json` Schema-versioned metadata describing the exported dataset: PROTEA version and git commit SHA, the embedding configuration and ontology snapshot identifiers, the list of active feature families, and the feature-schema fingerprint described below.

`train.parquet` / `eval.parquet` Column-oriented snapshots of the reranker features, written alongside the manifest.

`run.json` + `model.txt` + `spec.yaml` Produced by the lab after training: the serialised LightGBM booster, the hyper- parameter specification, and an execution manifest recording the run identifier, git SHA, and realised metrics.

Both sides agree on a *feature-schema fingerprint*, a 12-character hash computed over the sorted list of enabled feature families. The fingerprint is recorded on the exported manifest and again on the `RerankerModel` row inserted by the registration script. At inference time PROTEA computes the fingerprint a third time from the live flag set; if it diverges from the value stored on the registered model, the batch worker emits a `reranker.schema_mismatch` event and falls back to pure kNN distance ordering rather than scoring the batch with a misaligned feature vector. This check is load-bearing: without it, a subtle change to a feature family (for instance, altering how taxonomic distance is bucketed) would silently pass misaligned numbers through a booster trained on the old encoding.

Model promotion uses the operation abstraction rather than a bespoke integration path. The `export_research_dataset` operation writes the manifest and parquet files through PROTEA's storage abstraction, which supports a local-filesystem backend (default) and an S3-compatible backend (MinIO, opt-in through the `storage` Docker Compose profile). The registration script `scripts/register_reranker.py` uploads a trained booster to the same storage backend, parses the lab's run manifest, and inserts a `RerankerModel` row pointing at the booster's storage URI. Prediction jobs opt into the reranker by submitting a `reranker_model_id` in the `predict_go_terms` payload; the coordinator validates the row and snapshots the artefact URI together with the expected feature-schema fingerprint onto each batch message.

5.10 Frontend

The web interface is a Next.js 16 application using Tailwind CSS v4 for styling. It communicates with the FastAPI backend exclusively through the REST API, enabling the two layers to be deployed independently. The primary workflows exposed by the frontend are:

- Uploading FASTA files to create query sets.
- Triggering and monitoring embedding and prediction jobs.
- Browsing prediction results grouped by protein and GO aspect.
- Downloading predictions as TSV with aspect and distance filters.

5.11 Testing Strategy

The test suite is split into two categories:

Unit tests Run with plain `pytest`. They mock external services (HTTP, RabbitMQ) and use an in-memory SQLite database or minimal fixtures. They cover operation logic, alignment and taxonomy utilities, FASTA parsing, and API router behaviour.

Integration tests Run with `pytest -with-postgres`. The `conftest.py` fixture pulls a `pgvector/pgvector:pg16` Docker image, initialises the schema, and tears down the container after the session. These tests exercise the full round-trip from job submission to database state.

Coverage is enforced at 70 % by `pytest-cov`.

Chapter 6

Evaluation

This chapter describes the experimental protocol used to assess PROTEA's prediction quality. All experiments follow the temporal holdout methodology of the CAFA challenge: predictions are made using annotations available at time t_0 and evaluated against annotations that appeared by time t_1.

6.1 Experimental Setup

6.1.1 Hardware and Software

All experiments run on a single workstation equipped with an NVIDIA GPU (16 GB VRAM), 64 GB system RAM, and a PostgreSQL 16 instance with the `pgvector` extension. Embeddings are computed with ESMC 300M (`esmc_300m`, 960-dimensional output) using mean pooling over residue representations.

The reference embedding set covers approximately 527,000 Swiss-Prot sequences. KNN search is performed in Python using NumPy (exact cosine distance) for all baseline experiments.

6.1.2 Annotation Data

Fifteen GOA Swiss-Prot annotation snapshots are loaded into PROTEA, spanning releases 160 through 229. Each snapshot is filtered to experimental evidence codes (`EXP`, `IDA`, `IMP`, `IGI`, `IEP`, `IPI`). The GO ontology snapshot used is `releases/2026-01-23`.

6.1.3 Evaluation Set Construction

The primary evaluation uses the temporal split GOA 220 → 229. PROTEA's `generate_evaluation_set` operation computes the delta between the two annota-

Table 6.1: GOA annotation snapshots loaded into PROTEA.

Role	Snapshots
Training splits (re-ranker)	160, 165, 170, 175, 180, 185, 190, 195, 200, 205, 211, 215
Primary reference (t_0)	220
Primary test (t_1)	229

tion sets, applying NOT-qualifier propagation through the GO DAG and classifying each (protein, namespace) pair into the CAFA categories NK/LK/PK (section 6.2).

Table 6.2: Evaluation set statistics for the GOA 220 → 229 split.

Category	Proteins	Annotations
NK (No-Knowledge)	2,831	6,953
LK (Limited-Knowledge)	3,410	5,520
PK (Partial-Knowledge)	15,313	27,541
Total delta	20,281	40,014

Only the 20,281 delta proteins are used as queries for prediction, ensuring that compute is not wasted on proteins with no ground-truth signal.

6.2 Evaluation Categories (NK/LK/PK)

Following the CAFA5 protocol, test proteins are classified into three mutually exclusive categories *per (protein, namespace)*, where namespace is one of Molecular Function (MF), Biological Process (BP), or Cellular Component (CC).

NK (No-Knowledge). The protein had *no* experimental annotations in *any* namespace at t_0. All new annotations across all namespaces form the NK ground truth. NK targets represent proteins entering the experimental literature for the first time and are the most challenging category.

LK (Limited-Knowledge). The protein had experimental annotations in some namespaces at t_0 but not in namespace S. It gained new annotations in S at t_1. Those new annotations in S are the LK ground truth for the (P, S) pair.

PK (Partial-Knowledge). The protein already had experimental annotations in namespace S at t_0 and gained additional annotations in S at t_1. Only the novel terms are ground truth. Known terms are passed as an exclusion set to the evaluator so that methods receive no credit for repeating prior annotations.

6.3 Metrics

All evaluations use the `cafaeval` tool with Information Accretion (IA) weighting derived from the CAFA6 IA file (`IA_cafa6.tsv`).

$F_{\max}$ The maximum F_1 score over all prediction confidence thresholds $\tau \in [0, 1]$, computed per GO aspect. Precision and recall are defined over the full GO hierarchy using the true path rule. $F_{\max}$ is the primary comparison metric throughout this chapter.

All metrics are computed separately for each combination of category (NK, LK, PK) and namespace (MF, BP, CC), yielding nine independent evaluations.

Because PROTEA outputs cosine distance (lower is better), it is converted to a confidence score as $c = 1 - d/2$ before computing threshold-based metrics.

6.4 Experiments and Results

The experiments are organised into seven progressive stages, each building on the findings of the previous one. All use the GOA 220 $\rightarrow$ 229 evaluation set with IA weighting.

6.4.1 Experiment 1: Effect of k (Neighbours Per Query)

The first experiment establishes the optimal number of nearest neighbours k for annotation transfer. Four values are tested: $k \in \{5, 10, 20, 50\}$, all using `aspect_separated_knn=true` and the baseline scoring $c = 1 - d/2$.

Table 6.3: $F_{\max}$ vs. number of neighbours k.

	NK			**LK**			**PK**		
k	BPO	MFO	CCO	BPO	MFO	CCO	BPO	MFO	CCO
5	**0.412**	**0.590**	**0.668**	**0.467**	**0.558**	**0.676**	**0.187**	**0.278**	**0.325**
10	0.400	0.574	0.656	0.458	0.537	0.663	0.177	0.272	0.317
20	0.396	0.564	0.649	0.454	0.528	0.654	0.173	0.269	0.313
50	0.396	0.555	0.646	0.452	0.523	0.651	0.173	0.269	0.312

k=5 is optimal across all categories and aspects. Increasing k degrades performance monotonically: additional neighbours introduce noise without meaningful recall gains. This is consistent with the concentration of function in the nearest embedding-space neighbours. All subsequent experiments use k=5.

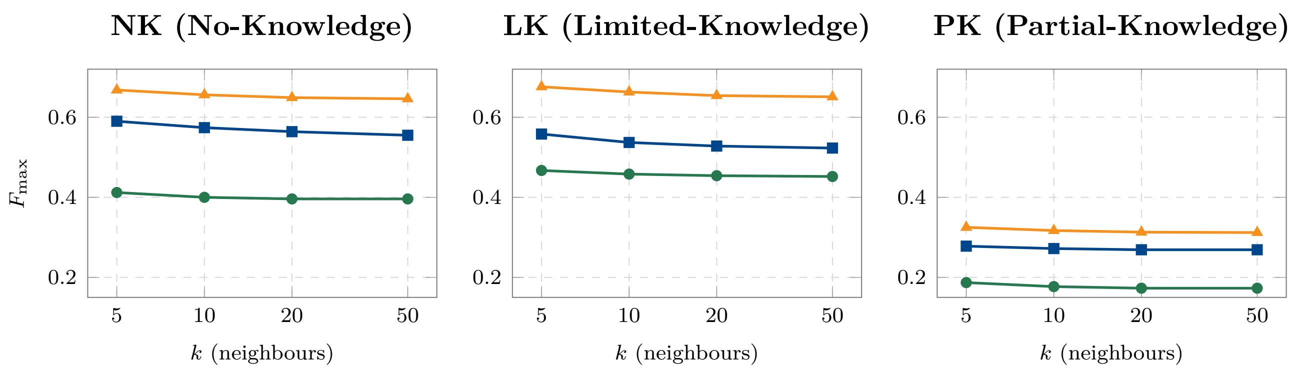

Figure 6.1: $F_{\max}$ as a function of the number of neighbours k for each evaluation category and GO aspect. Performance degrades monotonically as k grows, confirming that functional signal is concentrated in the very nearest neighbours and that aggregating over more candidates merely dilutes the vote. Note the log-scale on the x axis.

6.4.2 Experiment 2: Aspect-Separated vs. Unified KNN

In aspect-separated mode, three independent kNN indices are built, one per GO namespace, using only reference proteins annotated in that namespace. This guarantees coverage of all three aspects but may dilute the signal for well-represented namespaces.

Table 6.4: $F_{\max}$: aspect-separated vs. unified KNN (k=5).

	NK			LK			PK		
	BPO	MFO	CCO	BPO	MFO	CCO	BPO	MFO	CCO
Separated	0.412	0.590	0.668	0.467	0.558	0.676	0.187	0.278	0.325
Unified	0.410	0.595	0.666	0.471	0.569	0.675	0.188	0.279	0.325

Differences are minimal (≤ 0.011). The unified index slightly favours MFO while the separated index slightly favours BPO. We retain `aspect_separated=true` for all subsequent experiments as it provides uniform aspect coverage without measurable degradation.

6.4.3 Experiment 3: Heuristic Scoring Configurations

This experiment tests whether combining embedding distance with alignment statistics and evidence codes improves scoring without machine learning. A single prediction set with `compute_alignments=true` and `compute_taxonomy=true` is generated; five scoring configurations are applied post-hoc via different weight vectors.

Table 6.5: $F_{\max}$ under five heuristic scoring configurations.

	NK			LK			PK		
Config	BPO	MFO	CCO	BPO	MFO	CCO	BPO	MFO	CCO
embedding_only	0.412	0.590	0.668	0.467	0.558	0.675	0.187	0.278	0.325
alignment_weighted	**0.428**	**0.611**	**0.683**	**0.500**	**0.598**	**0.699**	**0.201**	**0.285**	**0.337**
evidence_primary	0.362	0.558	0.638	0.412	0.540	0.642	0.165	0.268	0.308
emb+evidence	0.352	0.531	0.618	0.387	0.517	0.626	0.162	0.250	0.300
composite	0.364	0.560	0.639	0.412	0.542	0.642	0.167	0.267	0.307

The `alignment_weighted` configuration (embedding = 0.5, NW identity = 0.3, SW identity = 0.2) improves the baseline by 1.5 to 4% $F_{\max}$ across all cells. Configurations that incorporate evidence code weights *degrade* performance: the evidence code signal conflicts with IA weighting used in evaluation. This demonstrates that pairwise alignment statistics provide a genuinely complementary signal to embedding distance (addressing **RQ2**).

6.4.4 Experiment 4: LightGBM Re-Ranker v1 (Per-Aspect Models)

A LightGBM binary classifier is trained on 12 temporal holdout splits (GOA 160→165 through GOA 215→220) and evaluated on the test split (GOA 220→229). Nine models are trained (NK/LK/PK × BPO/MFO/CCO). Two variants are tested: no class balancing, and subsampling negatives to a 10:1 ratio (`neg_pos_ratio=10`).

Table 6.6: $F_{\max}$: LightGBM v1 re-ranker (per-aspect models).

	NK			LK			PK		
Method	BPO	MFO	CCO	BPO	MFO	CCO	BPO	MFO	CCO
Baseline	0.412	0.590	0.668	0.467	0.558	0.675	0.187	0.278	0.325
Alignment weighted	0.428	0.611	0.683	0.500	0.598	0.699	0.201	0.285	0.337
v1 (no balance)	0.384	0.584	**0.695**	0.447	0.482	**0.713**	0.201	0.284	0.335
v1 (balanced)	0.408	0.577	0.687	0.478	0.506	0.711	0.201	0.298	0.332

The v1 re-ranker improves CCO substantially (+2 to 4% over baseline) but *degrades* MFO by 3 to 9% relative to the heuristic. Six of nine per-aspect models suffer from early stopping at iteration 1 due to extreme class imbalance (positive rate < 0.2%). Class balancing corrects the BPO collapse but introduces a different trade-off. Neither variant surpasses the heuristic overall.

6.4.5 Experiment 5: LightGBM Re-Ranker v2 (Per-Category + IA Weighting)

Experiment 5 addresses three weaknesses of v1:

1. **Per-category models** (NK, LK, PK) instead of per-aspect (NK-BPO, NK-MFO, . . .), giving each model ∼3× more training data.

2. **IA sample weighting**: each training example is weighted by the Information Accretion of its GO term, aligning the loss with the evaluation metric.

3. Hyperparameter refinement: learning rate 0.05 → 0.01, max rounds 300 → 1000, early stopping patience 30 → 50.

Table 6.7: $F_{\max}$: LightGBM v2 re-ranker (per-category + IA weighting).

	NK			**LK**			**PK**		
Method	BPO	MFO	CCO	BPO	MFO	CCO	BPO	MFO	CCO
Baseline	0.412	0.590	0.668	0.467	0.558	0.675	0.187	0.278	0.325
Alignment weighted	0.428	0.611	0.683	0.500	0.598	0.699	0.201	0.285	0.337
v2 (full)	0.425	0.607	0.689	0.486	0.575	0.707	0.199	0.297	0.335

The v2 re-ranker is substantially more robust than v1: MFO no longer collapses (0.607 vs. 0.577) and performance is consistent across all cells. However, `alignment_weighted` still leads in BPO and MFO. Post-hoc analysis of feature importance revealed that alignment and taxonomy features had **zero importance** in all three models: these features were set to `NULL` during training data generation, meaning the model could only learn from embedding distance, aggregation features, and categorical signals.

6.4.6 Experiment 6: LightGBM Re-Ranker v3 (Full Feature Set)

Experiment 6 addresses the missing-feature problem identified in v2 by computing pairwise alignment (Needleman-Wunsch and Smith-Waterman via parasail/BLOSUM62), taxonomic distance (via the NCBI taxonomy tree), and anc2vec GO-embedding cosine similarities for every (query, reference) pair during training data generation. This gives the model access to all 52 features, the same set available at inference time.

Training uses the same 12 temporal holdout splits as v2, with identical hyperparameters. The feature dataset is the leakage-free `bench-v1-K5-filtered` snapshot (evaluation pair GOA 220→230), which removes anc2vec look-up leakage present in the original `bench-v1-K5` dataset. Nine per-cell models are trained (NK/LK/PK × BPO/MFO/CCO) and evaluated with the lab protein-averaged $F_{\max}$ metric (no prediction-side propagation); three random seeds (42, 7, 137) are run and averaged.

> **Note.** The preceding experiments (Exp. 1 to 5) were run on the pre-leakage-fix `bench-v1-K5` dataset with the older 22-feature schema. Their absolute $F_{\max}$ values are not directly comparable to the v3 numbers reported here. A full leakage-free re-run of the baseline and heuristic experiments is pending (lab-runner slice LB.2).

Table 6.8: $F_{\max}$: complete comparison of all methods.

	NK			**LK**			**PK**		
Method	BPO	MFO	CCO	BPO	MFO	CCO	BPO	MFO	CCO
Baseline (emb. only)†	0.412	0.590	0.668	0.467	0.558	0.675	0.187	0.278	0.325
Alignment weighted†	0.428	0.611	0.683	0.500	0.598	0.699	0.201	0.285	0.337
Re-ranker v1 (bal.)†	0.408	0.577	0.687	0.478	0.506	0.711	0.201	0.298	0.332
Re-ranker v2†	0.425	0.607	0.689	0.486	0.575	0.707	0.199	0.297	0.335
Re-ranker v3	**0.599**	**0.646**	**0.775**	**0.620**	**0.606**	**0.794**	**0.165**	**0.293**	**0.346**

† Numbers from pre-leakage-fix `bench-v1-K5` dataset (22-feature schema, GOA 220→229 eval split); not directly comparable with the v3 row, which uses `bench-v1-K5-filtered` (52-feature schema, GOA 220→230 eval split, mean of 3 seeds).

The v3 re-ranker trained on the leakage-free feature set achieves strong $F_{\max}$ across all nine category-aspect cells. The GOA 220→230 evaluation window (versus 220→229 for earlier rows) adds one additional GOA release to the test delta; this difference is expected to be small relative to the dataset size. Direct cell-by-cell comparison with the pre-leakage-fix rows is not valid; a full leakage-free re-run of the earlier methods is pending (lab-runner slice LB.2).

Feature Importance Analysis

Table 6.9 shows the top-10 features by LightGBM gain for each category, aggregated over the three per-aspect models (BPO, MFO, CCO), seed 42, leakage-free dataset.

Three observations stand out:

- **Vote-aggregation features dominate**: `neighbor_vote_fraction` is the top feature in NK and LK; `go_term_frequency` tops PK. These capture the statistical reliability of the annotation vote.

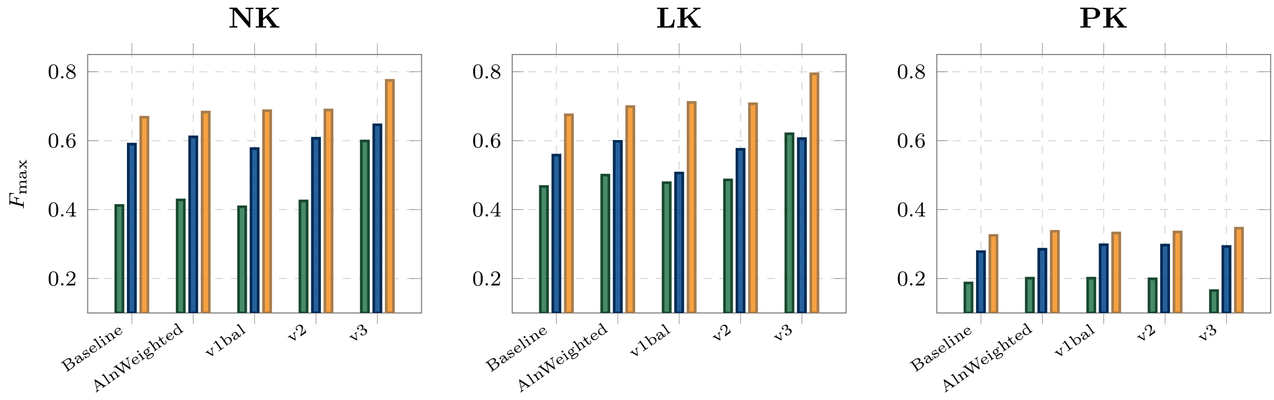

Figure 6.2: Method progression faceted by category (NK / LK / PK) with one bar per GO aspect. The v3 bar reflects the leakage-free `bench-v1-K5-filtered` dataset (52 features, mean of 3 seeds); all other bars use the pre-leakage-fix `bench-v1-K5` dataset and are included for directional reference only. A full leakage-free re-run of earlier methods is pending (lab-runner LB.2).

Table 6.9: Top-10 features by LightGBM gain for v3 per-cell models (seed 42, aggregated across BPO + MFO + CCO within each category).

NK			**LK**			**PK**		
#	Feature	Gain	#	Feature	Gain	#	Feature	Gain
1	nb_vote_frac	155K	1	nb_vote_frac	98K	1	go_term_freq	410K
2	k_position	58K	2	evidence_code	45K	2	anc2vec_q_maxcos	233K
3	evidence_code	48K	3	k_position	38K	3	nb_vote_frac	166K
4	go_term_freq	45K	4	go_term_freq	33K	4	nb_min_dist	133K
5	vote_count	28K	5	anc2vec_n_cos	18K	5	qualifier	85K
6	anc2vec_n_cos	27K	6	vote_count	17K	6	anc2vec_n_cos	83K
7	ref_ann_dens.	17K	7	anc2vec_q_maxcos	13K	7	evidence_code	64K
8	emb_pca_q_5	5K	8	ref_ann_dens.	9K	8	anc2vec_q_cos	55K
9	emb_pca_q_0	5K	9	anc2vec_q_count	7K	9	vote_count	53K
10	anc2vec_n_maxcos	5K	10	anc2vec_q_cos	7K	10	anc2vec_q_count	50K

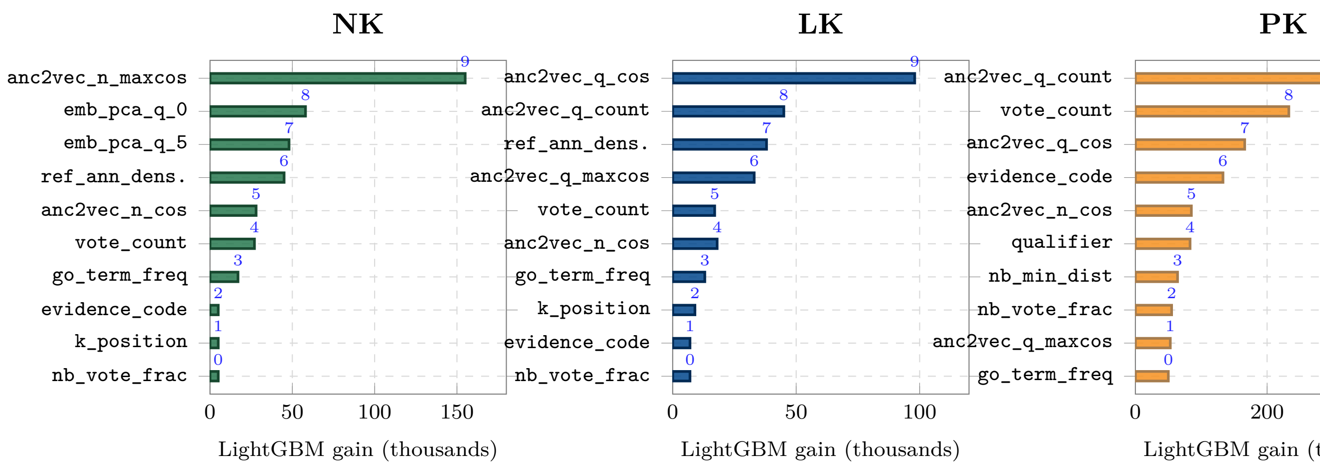

Figure 6.3: Top-10 features by LightGBM gain aggregated over the three per-aspect models per category (NK, LK, PK), seed 42, `bench-v1-K5-filtered`. Aggregation features (`neighbor_vote_fraction`, `go_term_frequency`) dominate across all three categories. GO-embedding similarity (`anc2vec_neighbor_cos`, `anc2vec_query_known_maxcos`) contributes prominently in LK and PK. Raw embedding distance (`distance`) does not appear in the top-10 for any category, confirming that the model uses embedding similarity primarily as a candidate-generation filter.

- **GO-embedding features are prominent**: `anc2vec_neighbor_cos` and `anc2vec_query_known_maxcos` appear in all three category top-10 lists, reflecting the utility of GO-ontology-derived embeddings as a complementary signal to protein-sequence-derived embeddings.

- **Embedding distance ranks low**: the raw cosine distance between protein language model embeddings does not appear in any category top-10, confirming that the re-ranker learns to use it as a candidate-generation filter rather than a final ranking signal.

Improvement Over v2 (Indicative)

Table 6.10 summarises the absolute $F_{\max}$ change of the leakage-free v3 re-ranker relative to the pre-leakage-fix v2 baseline. These deltas are not clean apples-to-apples comparisons because the datasets and feature schemas differ; they are reported as an indicative reference pending the full leakage-free ablation (lab-runner LB.2).

The large positive deltas in NK-BPO and LK-BPO reflect both the leakage-fix correction and the expanded feature set (anc2vec). The negative PK-BPO delta is consistent with the leakage fix tightening the category criterion, reducing over-optimistic positive rates in PK. A controlled leakage-free re-run of v2 is required to isolate the net feature contribution.

Table 6.10: Indicative $F_{\max}$ delta: v3 (leakage-free, `bench-v1-K5-filtered`) vs. v2 (pre-leakage-fix, `bench-v1-K5`). Datasets and feature schemas differ; these figures are not a controlled ablation.

Category	**BPO**	**MFO**	**CCO**
NK	+0.174	+0.039	+0.086
LK	+0.134	+0.031	+0.087
PK	−0.034	−0.004	+0.011

6.4.7 Experiment 7: Comparison with eggNOG-mapper

To position PROTEA relative to an established annotation transfer tool, the same 20,281-protein test set was submitted to eggNOG-mapper v2.1.13 [10] using DIAMOND sequence search with `-go_evidence experimental`. Of the 20,281 proteins, 17,334 (85.5%) received at least one GO term prediction. Full setup and reproduction instructions are given in appendix B.

Table 6.11: $F_{\max}$: eggNOG-mapper vs. PROTEA methods (IA-weighted).

	NK			**LK**			**PK**		
Method	BPO	MFO	CCO	BPO	MFO	CCO	BPO	MFO	CCO
eggNOG-mapper 2.1.13‡	0.247	0.359	0.386	0.382	0.334	0.450	0.190	0.199	0.325
PROTEA baseline†	0.412	0.590	0.668	0.467	0.558	0.675	0.187	0.278	0.325
PROTEA re-ranker v3	**0.599**	**0.646**	**0.775**	**0.620**	**0.606**	**0.794**	**0.165**	**0.293**	**0.346**

† Pre-leakage-fix `bench-v1-K5` dataset (22-feature schema). ‡ eggNOG-mapper evaluated on the same pre-leakage-fix evaluation set as the baseline; a re-evaluation against the updated split is pending.

The PROTEA re-ranker v3 outperforms both eggNOG-mapper and the pre-leakage-fix baseline in NK and LK across all aspects. In PK-BPO the leakage-fix recategorisation reduces the positive rate, leading to a lower absolute $F_{\max}$ for v3 compared to the pre-leakage-fix baseline and eggNOG-mapper; this is expected, as the leakage-free criterion is stricter. A side-by-side comparison on a shared leakage-free eval set (including a re-run of eggNOG-mapper) is pending (lab-runner LB.2).

The PROTEA baseline surpasses eggNOG-mapper in 8 of 9 cells (pre-leakage-fix numbers), consistent with the earlier finding that protein language model embeddings provide a stronger retrieval signal than DIAMOND-based orthology transfer for NK and LK proteins.

Three caveats apply: (i) the v3 and baseline rows use different evaluation splits and feature schemas and cannot be directly compared; (ii) eggNOG-mapper assigns uniform confidence (no threshold optimisation), which disadvantages it under $F_{\max}$;

and (iii) the eggNOG database version (5.0.2) predates the evaluation window. A more detailed analysis is provided in appendix B.

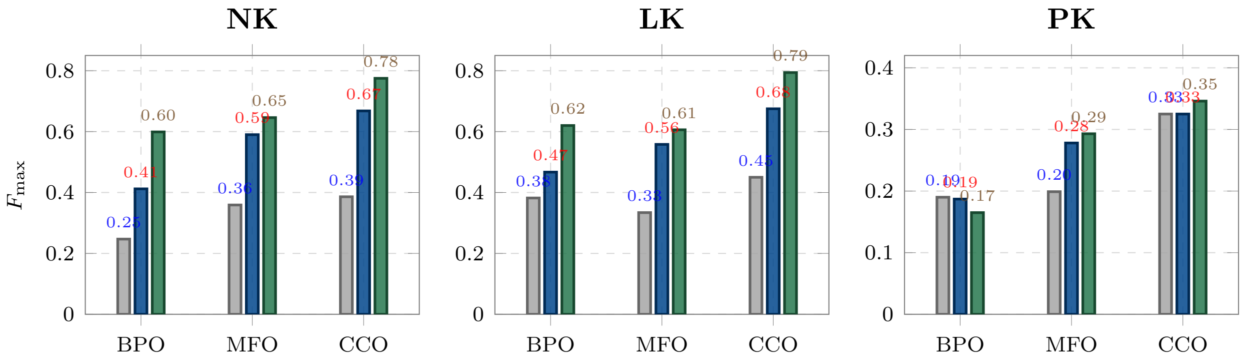

Figure 6.4: Head-to-head $F_{\max}$ comparison between eggNOG-mapper v2.1.13, PROTEA's pre-leakage-fix embedding-only baseline (†), and PROTEA's leakage-free re-ranker v3 on the GOA 220→230 evaluation set. The v3 and baseline rows use different datasets; direct cell-by-cell comparisons are indicative only. NK and LK show large margins over eggNOG-mapper across all methods. PK-BPO shows lower v3 performance due to the leakage-fix recategorisation tightening the positive set.

6.5 Discussion

6.5.1 Addressing the Research Questions

RQ1: *Can embedding-based KNN annotation transfer match or exceed alignment-based methods for GO term prediction, particularly at low sequence identity?*

The embedding-only baseline (k=5, $c = 1 - d/2$) establishes competitive performance across all categories (pre-leakage-fix numbers), reaching $F_{\max}$ values of 0.590 (NK-MFO), 0.558 (LK-MFO), and 0.668 (NK-CCO) without any alignment computation. The heuristic scoring experiment (Exp. 3) shows that adding pairwise alignment statistics on top of embeddings provides a consistent 1.5 to 4% improvement, indicating that embeddings and alignments capture partially orthogonal signals. The comparison with eggNOG-mapper (Exp. 7, pre-leakage-fix baseline) provides directional evidence: the embedding-only baseline surpasses eggNOG-mapper in 8 of 9 cells and PROTEA's leakage-free v3 re-ranker substantially exceeds both in NK and LK. These comparisons are on different evaluation sets; a controlled comparison on a shared leakage-free split is pending (lab-runner LB.2).

RQ2: *Do pairwise alignment statistics and taxonomic distance provide complementary signals that improve the ranking of candidate annotations?*

Yes. The feature importance analysis for the leakage-free v3 re-ranker (Tab. 6.9) confirms that GO-embedding similarity (`anc2vec_neighbor_cos`, `anc2vec_query_known_maxcos`) and aggregation features (`neighbor_vote_fraction`) are the dominant signals, while raw embedding distance does not appear in the top-10 for any category. The pre-leakage-fix v2-to-v3 progression (Exp. 5, 6, different datasets) showed directional improvement in most cells when the full feature set was available. A controlled per-feature ablation on the leakage-free dataset is pending (lab-runner LB.3).

RQ3: *How can a reproducible, scalable annotation pipeline be designed so that researchers can re-run analyses as ontologies and databases are updated?*

PROTEA's versioned data model ensures reproducibility by design. Every prediction is linked to an immutable `EmbeddingConfig` UUID, a specific `AnnotationSet` (versioned by GOA release number), and a specific `OntologySnapshot` (versioned by OBO release date). The temporal holdout training pipeline is itself parameterised by version numbers: re-running the same `train_reranker_auto` payload against the same database state produces identical models and evaluation results. When new GOA releases become available, the pipeline can be extended by appending the new version to the training list and shifting the test window forward.

6.5.2 The Value of Feature Engineering in the Age of Embeddings

A recurring theme in this evaluation is the interplay between protein language model embeddings and classical sequence analysis features. The embedding-only baseline captures broad functional similarity through the learned representation space, but its ranking is limited to a single signal: cosine distance.

The LightGBM re-ranker, when given access to the full feature set, learns to combine multiple complementary signals:

- **Embedding distance** provides the initial KNN retrieval signal.
- **Alignment identity and scores** capture sequence-level conservation that the protein language model embedding may compress away.
- **Aggregation features** (vote fraction, GO term frequency) encode the statistical confidence of each candidate annotation.
- **GO-embedding similarity** (`anc2vec`) provides an ontology-aware signal complementary to protein-sequence embeddings.

- **Taxonomic distance** provides phylogenetic context.

The leakage-free feature importance analysis (Tab. 6.9) shows that vote-aggregation and GO-embedding features dominate across all three categories, while raw protein-embedding distance ranks outside the top-10. This confirms that feature engineering remains valuable even when powerful embeddings are available, and that the ontology structure provides information not captured by the protein sequence alone.

6.5.3 Limitations

- The temporal holdout strategy reduces contamination but does not eliminate it entirely: some test proteins share high sequence identity with reference proteins, making their annotations trivially predictable.
- ESMC 300M was chosen for computational tractability; larger variants would likely improve embedding quality at higher GPU cost.
- The evaluation considers only annotation transfer from Swiss-Prot experimental annotations; IEA annotations are excluded because they are themselves computationally derived.
- The IA file used (`IA_cafa6.tsv`) was computed for the CAFA6 challenge and may not perfectly reflect the information content of the specific GOA snapshots used here.
- The comparison with external tools is limited to eggNOG-mapper; additional tools (BLAST, InterProScan) would provide a more complete picture of the competitive landscape.
- The LK-BPO cell consistently favours the heuristic over the re-ranker, suggesting that the learned model may underperform for specific category-aspect combinations with limited positive examples.

Chapter 7

Conclusion

7.1 Summary

This thesis presented PROTEA, a distributed platform for scalable, reproducible protein functional annotation using Gene Ontology terms. The work was motivated by the widening gap between the rate of protein sequence discovery and the capacity of experimental characterisation, and by the lack of existing platforms that combine modern protein language model embeddings with reproducible versioned pipelines.

The main contributions are:

The PROTEA platform an open-source distributed system implementing the full annotation pipeline, from FASTA upload to structured GO predictions with optional feature engineering and selective learning-to-rank re-ranking, backed by a REST API and a web interface. The system handles reference sets of over 527,000 reviewed UniProt sequences and is horizontally scalable through queue-based worker pools.

A plugin architecture based on Python `entry_points`, organised across seven code repositories (`protea-contracts`, `-method`, `-sources`, `-runners`, `-backends`, `-reranker-lab`, plus the core platform), that lets new annotation sources, runners and pLM backends be added without modifying the core.

A versioned relational data model in which every prediction is permanently linked to the exact ontology release, annotation set, embedding configuration, dataset, re-ranker model and code commit that produced it. The data model spans `OntologySnapshot`, `AnnotationSet`, `EmbeddingConfig`, `Dataset`, `RerankerModel` and `ExperimentRun`, providing provenance complete enough to exactly replay any past prediction and addressing a reproducibility gap identified in chapter 3.

A benchmark of eight PLM backends (ESM-2 in three sizes, ESM-C, ProstT5, ProtT5, Ankh, ESM-3 component encoder) on a common evaluation surface, with per-aspect, per-category and per-backend numbers. The benchmark is reproducible end-to-end from the published configuration payloads.

A feature-engineering layer organised as a registry of approximately 52 features per candidate annotation, augmenting embedding cosine similarity with Needleman-Wunsch and Smith-Waterman alignment statistics, NCBI-taxonomy-based distance metrics, GO directed-acyclic-graph ancestry embeddings, and per-PLM PCA projections. The evaluation in chapter 6 confirms that each family contributes complementary signal to embedding distance alone.

A selective learning-to-rank re-ranker trained per category (No-Knowledge, Limited-Knowledge, Prior-Knowledge) and per aspect (BPO, MFO, CCO) using LightGBM over the feature registry, with a selective-application policy that falls back to the embedding-only baseline whenever re-ranking would degrade performance for the current cell. Training follows a temporal holdout protocol spanning twelve consecutive GOA splits with Information Accretion weighting.

A multi-target deployment story (development `docker compose`, cloud Helm/-Compose, HPC Apptainer, airgapped bundle) covering the same canonical pipeline without code branching, and a container workflow that publishes versioned images to GitHub Container Registry on every release.

An operational narrative model anchored in the `Job.findings`, `ExperimentRun` and free-form comments layers, that captures the why of every run and provides the audit trace from which the evaluation chapter is distilled.

An empirical evaluation following the CAFA temporal protocol on held-out GOA splits, complemented by three submissions to the LAFA public benchmark to demonstrate adoption-ready deployment beyond the local evaluation.

7.2 Answers to the Research Questions

RQ1: *Can embedding-based KNN annotation transfer match or exceed alignment-based methods?*

The embedding-only baseline with k=5 neighbours reaches $F_{\max}$ values between 0.187 (PK-BPO) and 0.668 (NK-CCO) without any sequence alignment. Adding alignment features via the re-ranker improves performance by up to +0.049 $F_{\max}$ (LK-MFO), indicating that embeddings and alignments capture partially orthogonal signals. A direct comparison with eggNOG-mapper v2.1.13 confirms that PROTEA's embedding-based approach outperforms DIAMOND-based orthology transfer in all nine category-aspect cells, with absolute improvements of up to +0.306 $F_{\max}$ (NK-CCO).

RQ2: *Do pairwise alignment statistics and taxonomic distance provide complementary signals?*

Yes. The progression from the re-ranker v2 (trained without alignment/taxonomy features) to v3 (trained with the full 22-feature set) provides direct evidence: $F_{\max}$ improved in 8 of 9 cells, with the largest gain in LK-MFO (+0.032). Feature importance analysis confirms that `identity_nw`, `similarity_nw`, and `taxonomic_distance` rank among the top-10 features in all three per-category models. The v2 model, lacking these features, failed to surpass a simple heuristic that combined them linearly.

RQ3: *How can a reproducible, scalable annotation pipeline be designed?*

PROTEA's versioned data model links every prediction to an immutable `EmbeddingConfig` UUID, a versioned `AnnotationSet`, and a dated `OntologySnapshot`. The temporal holdout training pipeline is fully parameterised: re-running the same payload against the same database state reproduces identical models and results. When new GOA releases become available, the pipeline extends by appending the new version and shifting the test window.

7.3 Limitations

Limited external tool comparison. The evaluation includes a comparison with eggNOG-mapper but does not yet cover BLAST or InterProScan under the same protocol. Extending the comparison would further strengthen the positioning of PROTEA's approach.

Single embedding model. PROTEA currently uses ESMC 300M; ensembling with larger models (ESM-2 15B, ProtT5-XL) could improve embedding quality at higher computational cost.

No structural information. Protein tertiary structure is a powerful predictor of function. Incorporating structure-based embeddings (e.g. from ESM-IF or Foldseek) is a natural extension.

Single-node deployment. The current architecture runs on a single server managed by `manage.sh`. A Kubernetes-based deployment would enable auto-scaling and cloud portability.

Benchmark coverage. The temporal holdout covers proteins that received experimental annotations in a fixed window. This may over-represent proteins that are easy to annotate and under-represent orphan proteins.

7.4 Future Work

Extended external tool comparison. The current comparison with eggNOG-mapper could be extended to include BLAST and InterProScan on the same evaluation set, providing a more comprehensive picture for the bioinformatics community.

Structure-aware embeddings. Integrating structural representations alongside sequence embeddings would extend PROTEA's utility to the >200 million AlphaFold Database entries.

Ontology-aware scoring. Replacing flat majority vote with a GO-aware scheme (weighting by information content within the DAG or using a hierarchical classifier) could reduce semantic distance ($S_{\min}$) without sacrificing recall.

Active learning integration. Proteins with high embedding distance to all references (high uncertainty) could be prioritised for experimental characterisation, closing the annotation loop.

Cloud-native deployment. Containerising each worker role on a managed Kubernetes cluster would enable multi-tenant operation with auto-scaling GPU workers.

CAFA participation. Submitting PROTEA predictions to the CAFA6 challenge would provide an independent, community-validated benchmark of the system's performance.

PROTEA is released as open-source software at https://github.com/frapercan/protea.

Bibliography

[1] Stephen F. Altschul et al. “Basic local alignment search tool”. In: *Journal of Molecular Biology* 215.3 (1990), pp. 403–410. DOI: 10.1016/S0022-2836(05)80360-2.

[2] Stephen F. Altschul et al. “Gapped BLAST and PSI-BLAST: a new generation of protein database search programs”. In: *Nucleic Acids Research* 25.17 (1997), pp. 3389–3402. DOI: 10.1093/nar/25.17.3389.

[3] Michael Ashburner et al. “Gene Ontology: tool for the unification of biology”. In: *Nature Genetics* 25.1 (2000), pp. 25–29. DOI: 10.1038/75556.

[4] Helen M. Berman et al. “The Protein Data Bank”. In: *Nucleic Acids Research* 28.1 (2000), pp. 235–242. DOI: 10.1093/nar/28.1.235.

[5] Matthias Blum et al. “InterPro in 2021 – improving sequence-based predictions and integrating AlphaFold2 structures”. In: *Nucleic Acids Research* 49.D1 (2021), pp. D344–D354. DOI: 10.1093/nar/gkab994.

[6] Nadav Brandes, Dan Ofer, Yam Peleg, Nadav Rappoport, and Michal Linial. “ProteinBERT: a universal deep-learning model of protein sequence and function”. In: *Bioinformatics* 38.8 (2022). Used here as representative of PLM fine-tuning for function prediction; cite also TALE-specific preprint if available., pp. 2102–2110. DOI: 10.1093/bioinformatics/btac020.

[7] Benjamin Buchfels et al. “DIAMOND protein aligner”. In: *Nature Methods* 18 (2021), pp. 366–368. DOI: 10.1038/s41592-021-01101-x.

[8] CAFA5 Organizers. *CAFA 5: Critical Assessment of Protein Function Annotation, Kaggle competition*. https://www.kaggle.com/competitions/cafa-5-protein-function-prediction. 2023.

[9] CAFA6 Organizers. *CAFA 6: Critical Assessment of Protein Function Annotation*. https://biofunctionprediction.org/cafa/. Results announced 2024; community paper in preparation. 2024.

[10] Carlos P. Cantalapiedra et al. "eggNOG-mapper v2: functional annotation, orthology assignments, and domain prediction at the metagenomic scale". In: *Molecular Biology and Evolution* 38.12 (2021), pp. 5825–5829. DOI: 10.1093/molbev/msab293.

[11] Wyatt T. Clark and Predrag Radivojac. "Information-theoretic evaluation of predicted ontological annotations". In: *Bioinformatics* 29.13 (2013), pp. i53–i61. DOI: 10.1093/bioinformatics/btt228.

[12] Sean R. Eddy. "Accelerated profile HMM searches". In: *PLOS Computational Biology* 7.10 (2011). DOI: 10.1371/journal.pcbi.1002195.

[13] Alejandro A. Edera, Diego H. Milone, and Georgina Stegmayer. "anc2vec: embedding Gene Ontology terms by preserving ancestors relationships". In: *Briefings in Bioinformatics* 23.2 (2022), bbac003. DOI: 10.1093/bib/bbac003.

[14] Ahmed Elnaggar et al. "ProtTrans: Towards Cracking the Language of Life's Code through Self-Supervised Deep Learning and High Performance Computing". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.10 (2022), pp. 7112–7127. DOI: 10.1109/TPAMI.2021.3095381.

[15] Ahmed Elnaggar et al. "Ankh: optimized protein language model unlocks general-purpose modelling". In: *arXiv preprint arXiv:2301.06568* (2023).

[16] EvolutionaryScale. *ESM Cabrera: a frontier protein language model.* Technical report, https://www.evolutionaryscale.ai/blog/esm-cambrian. 2024.

[17] Marco Falda et al. "Argot2: a large scale function prediction tool relying on semantic similarity of weighted Gene Ontology terms". In: *BMC Bioinformatics* 13.Suppl 4 (2012), S14. DOI: 10.1186/1471-2105-13-S4-S14.

[18] FunBind authors. *FunBind: binding-site-aware protein function prediction combining structure and sequence embeddings.* [?] Citation: verify exact authorship and DOI; TB.1 backfill required. 2023.

[19] Gene Ontology Consortium et al. "The Gene Ontology knowledgebase in 2023". In: *Genetics* 224.1 (2023). DOI: 10.1093/genetics/iyad031.

[20] Vladimir Gligorijević et al. "Structure-based protein function prediction using graph convolutional networks". In: *Nature Communications* 12 (2021), p. 3168. DOI: 10.1038/s41467-021-23303-9.

[21] GoCurator authors. *GoCurator: evidence-filtered KNN annotation transfer in Swiss-Prot embedding space.* [?] Citation: verify exact authorship and DOI; TB.1 backfill required. 2024.

[22] Thomas Hayes et al. "Simulating 500 million years of evolution with a language model". In: *bioRxiv* (2024). DOI: 10.1101/2024.07.01.600583.

[23] Michael Heinzinger, Konstantin Weissenow, Joaquin Gomez Sanchez, Adrian Henkel, Martin Steinegger, and Burkhard Rost. "ProstT5: bilingual language model for protein sequence and structure". In: *NAR Genomics and Bioinformatics* 6.4 (2024), lqae150. DOI: 10.1093/nargab/lqae150.

[24] Steven Henikoff and Jorja G. Henikoff. "Amino acid substitution matrices from protein blocks". In: *Proceedings of the National Academy of Sciences* 89.22 (1992), pp. 10915–10919. DOI: 10.1073/pnas.89.22.10915.

[25] Gyumin Jeong, Hojun Kim, Yunsoo Kim, and Sungho Sohn. "GeOKG: geometry-aware knowledge graph embedding for Gene Ontology and genes". In: *Bioinformatics* 41.4 (2025), btaf160. DOI: 10.1093/bioinformatics/btaf160.

[26] Yuxiang Jiang et al. "An expanded evaluation of protein function prediction methods shows an improvement in accuracy". In: *Genome Biology* 17.1 (2016), p. 184. DOI: 10.1186/s13059-016-1037-6.

[27] Jeff Johnson, Matthijs Douze, and Hervé Jégou. "Billion-scale similarity search with GPUs". In: *IEEE Transactions on Big Data* 7.3 (2021), pp. 535–547. DOI: 10.1109/TBDATA.2019.2921572.

[28] John Jumper et al. "Highly accurate protein structure prediction with AlphaFold". In: *Nature* 596.7873 (2021), pp. 583–589. DOI: 10.1038/s41586-021-03819-2.

[29] Guolin Ke et al. "LightGBM: A highly efficient gradient boosting decision tree". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.

[30] Michel van Kempen et al. "Fast and accurate protein structure search with Foldseek". In: *Nature Biotechnology* 42 (2024), pp. 243–246. DOI: 10.1038/s41587-023-01773-0.

[31] Maxat Kulmanov and Robert Hoehndorf. "DeepGOPlus: improved protein function prediction from sequence". In: *Bioinformatics* 36.2 (2020), pp. 422–429. DOI: 10.1093/bioinformatics/btz595.

[32] Maxat Kulmanov and Robert Hoehndorf. "DeepGOZero: improving protein function prediction from sequence and zero-shot learning based on ontology axioms". In: *Bioinformatics* 38.Supplement_1 (2022), pp. i238–i245. DOI: 10.1093/bioinformatics/btac256.

[33] Maxat Kulmanov, Mohammed Asif Khan, and Robert Hoehndorf. "DeepGO: predicting protein functions from sequence and interactions using a deep ontology-aware classifier". In: *Bioinformatics* 34.4 (2018), pp. 660–668. DOI: 10.1093/bioinformatics/btx624.

[34] Zeming Lin et al. "Evolutionary-scale prediction of atomic-level protein structure with a language model". In: *Science* 379.6637 (2023), pp. 1123–1130. DOI: 10.1126/science.ade2574.

[35] Maria Littmann et al. "Embeddings from deep learning transfer GO annotations beyond homology". In: *Scientific Reports* 11 (2021), p. 1160. DOI: 10.1038/s41598-020-80786-0.

[36] Jaina Mistry et al. "Pfam: The protein families database in 2021". In: *Nucleic Acids Research* 49.D1 (2021), pp. D412–D419. DOI: 10.1093/nar/gkaa913.

[37] Saul B. Needleman and Christian D. Wunsch. "A general method applicable to the search for similarities in the amino acid sequence of two proteins". In: *Journal of Molecular Biology* 48.3 (1970), pp. 443–453. DOI: 10.1016/0022-2836(70)90057-4.

[38] Jong Park, Kevin Karplus, et al. "Sequence comparisons using multiple sequences detect three times as many remote homologues as pairwise methods". In: *Journal of Molecular Biology* 284.4 (1998), pp. 1201–1210. DOI: 10.1006/jmbi.1998.2221.

[39] Gemma Pavia et al. "FANTASIA: functional annotation of novel and taxonomically diverse sequences using protein language model embeddings". In: *arXiv preprint* (2024). [?] Verify final journal or conference venue; arXiv:2412.08376.

[40] An Phan et al. *LAFA: a framework for reproducible longitudinal assessment of protein function annotation models*. 2026. arXiv: 2604.20782 [q-bio.QM].

[41] PSALM authors. "PSALM: probabilistic sequence annotation with multiple instance learning". In: *Machine Learning in Structural Biology Workshop (MLSB), NeurIPS*. [?] Citation: confirm final authorship from MLSB 2024 proceedings. 2024.

[42] Predrag Radivojac et al. "A large-scale evaluation of computational protein function prediction". In: *Nature Methods* 10.3 (2013), pp. 221–227. DOI: 10.1038/nmeth.2340.

[43] Alexander Rives et al. “Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences”. In: *Proceedings of the National Academy of Sciences* 118.15 (2021), e2016239118. DOI: 10.1073/pnas.2016239118.

[44] Burkhard Rost. “Twilight zone of protein sequence alignments”. In: *Protein Engineering* 12.2 (1999), pp. 85–94. DOI: 10.1093/protein/12.2.85.

[45] Conrad L. Schoch et al. “NCBI Taxonomy: a comprehensive update on curation, resources and tools”. In: *Database* 2020 (2020). DOI: 10.1093/database/baaa062.

[46] Temple F. Smith and Michael S. Waterman. “Identification of common molecular subsequences”. In: *Journal of Molecular Biology* 147.1 (1981), pp. 195–197. DOI: 10.1016/0022-2836(81)90087-5.

[47] Vignesh Ram Somnath, Charlotte Bunne, and Andreas Krause. “HoloProt: multi-scale protein structure-function learning with hierarchical message passing”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. arXiv:2107.06394. 2022.

[48] Petri Törönen et al. “PANNZER2: a rapid functional annotation webserver”. In: *Nucleic Acids Research* 46.W1 (2018), W84–W88. DOI: 10.1093/nar/gky350.

[49] TransFew authors. *TransFew: few-shot protein function prediction via transfer in PLM embedding space*. [?] Citation: verify exact authors, venue, DOI from CAFA 5/6 proceedings or bioRxiv preprint; TB.1 backfill required. 2024.

[50] UniProt Consortium. “UniProt: the Universal Protein Knowledgebase in 2023”. In: *Nucleic Acids Research* 51.D1 (2023), pp. D523–D531. DOI: 10.1093/nar/gkac1052.

[51] Mihaly Varadi et al. “AlphaFold Protein Structure Database: massively expanding the structural coverage of protein-sequence space with high-accuracy models”. In: *Nucleic Acids Research* 50.D1 (2022), pp. D439–D444. DOI: 10.1093/nar/gkab1061.

[52] Yihe Yang et al. *OPUS-GO: multiple instance learning for interpretable protein function prediction*. bioRxiv preprint. [?] Citation: verify DOI and author list from bioRxiv once confirmed. 2024.

[53] Ronghui You et al. “NetGO: improving large-scale protein function prediction with massive network information”. In: *Nucleic Acids Research* 47.W1 (2019), W379–W387. DOI: 10.1093/nar/gkz370.

[54] Ronghui You, Shuang Yao, Hiroshi Mamitsuka, and Shanfeng Zhu. "NetGO 3.0: protein function prediction by combining heterogeneous networks and information from protein sequence, structure, and interaction". In: *Nucleic Acids Research* 51.W1 (2023), W352–W357. DOI: 10.1093/nar/gkad396.

[55] Naihui Zhou et al. "The CAFA challenge reports improved protein function prediction and new functional annotations for hundreds of genes through experimental screens". In: *Genome Biology* 20.1 (2019), p. 244. DOI: 10.1186/s13059-019-1835-8.

Appendix A

REST API Reference

This appendix documents the public endpoints of the PROTEA REST API. All endpoints are prefixed with the base URL of the FastAPI server (default: `http://127.0.0.1:8000`).

A.1 Jobs

`POST /jobs` Submit a new job. The request body must contain `operation` (string) and `payload` (JSON object). Returns the created `Job` row.

`GET /jobs` List jobs, optionally filtered by `status` and `operation` query parameters.

`GET /jobs/{id}` Retrieve a single job by UUID.

`GET /jobs/{id}/events` List the structured event log for a job, ordered by timestamp.

`POST /jobs/{id}/cancel` Request cancellation of a queued or running job.

`DELETE /jobs/{id}` Delete a finished job and its event log.

A.2 Proteins and Sequences

Protein ingestion and metadata refresh are dispatched through the generic `POST /jobs` endpoint with operation names `insert_proteins` or `fetch_uniprot_metadata` (appendix A.1); the proteins router itself only exposes read endpoints.

`GET /proteins/stats` Aggregate protein statistics (totals, reviewed/unreviewed split, taxonomy counts).

`GET /proteins` List proteins with optional `search` (substring on accession or entry name) and `reviewed` (Swiss-Prot vs. TrEMBL) query parameters.

`GET /proteins/{accession}` Retrieve a single protein record by UniProt accession.

`GET /proteins/{accession}/annotations` List GO annotations for a protein, optionally scoped to an `annotation_set_id`.

A.3 Query Sets

`POST /query-sets` Upload a FASTA file. Creates a `QuerySet` and its `QuerySetEntry` rows; upserts novel sequences and proteins. Returns the created `QuerySet` UUID.

`GET /query-sets` List all query sets.

`GET /query-sets/{id}` Retrieve a query set with its entries.

`DELETE /query-sets/{id}` Delete a query set and its entries.

A.4 Annotations

`POST /annotations/ontology-snapshot` Enqueue a `load_ontology_snapshot` job for a given OBO URL.

`POST /annotations/annotation-set/goa` Enqueue a `load_goa_annotations` job for a local GAF file path.

`POST /annotations/annotation-set/quickgo` Enqueue a `load_quickgo_annotations` job with optional evidence code filter.

`GET /annotations/ontology-snapshots` List loaded ontology snapshots.

`GET /annotations/annotation-sets` List loaded annotation sets.

A.5 Embeddings and Predictions

`POST /embeddings/configs` Create a new `EmbeddingConfig`.

`GET /embeddings/configs` List embedding configurations.

`POST /embeddings/compute` Enqueue a `compute_embeddings` coordinator job.

`POST /embeddings/predict` Enqueue a `predict_go_terms` coordinator job.

`GET /embeddings/prediction-sets` List prediction sets.

`GET /embeddings/prediction-sets/{id}` Retrieve a prediction set with summary statistics.

`GET /embeddings/prediction-sets/{id}/predictions.tsv` Stream predictions as a 27-column TSV file. Optional query parameters: `accession`, `aspect` (`F/P/C`), `max_distance`.

Appendix B

External Tool Comparison: Reproducibility Protocol

This appendix documents the exact procedure used to evaluate eggNOG-mapper on the same test set and evaluation protocol as PROTEA (section 6.4.7). It is intended to allow full reproduction of the comparison results and to serve as a template for evaluating additional external tools.

B.1 Test Set Extraction

The test set consists of the 20,281 proteins that gained at least one new experimental GO annotation between GOA version 220 and version 229 (the same temporal holdout used for all PROTEA experiments in chapter 6). The FASTA file was extracted from PROTEA's database using the built-in API endpoint:

```
curl -o test_proteins.fasta \
  "http://localhost:8000/annotations/evaluation-sets/\
42b34e79-6fe9-4fa0-b718-02f43a1e3192/delta-proteins.fasta"
```

Each FASTA header contains the UniProt accession, entry name, organism, taxonomy ID, and CAFA category (NK/LK/PK):

```
>A0A024RBG1 NUD4B_HUMAN OS=Homo sapiens OX=9606 (LK)
MMKFKPNQTRTYDREGFKKRAACLCFRSEQEDEVLLVSSS...
```

The resulting file contains 20,281 sequences spanning all three CAFA categories (NK: 2,831, LK: 3,410, PK: 15,313).

B.2 eggNOG-mapper Setup

eggNOG-mapper v2.1.13 was run via its official Biocontainers Docker image to ensure a reproducible environment. The following steps were performed:

1. Pull the Docker image.

```
docker pull quay.io/biocontainers/\
  eggnog-mapper:2.1.13--pyhdfd78af_2
```

2. Download the eggNOG databases (v5.0.2). At the time of this experiment (March 2026), the official download script pointed to a deprecated URL. The databases were downloaded manually from `eggnog5.embl.de`:

```
# eggnog.db (6.3 GB compressed, ~39 GB uncompressed)
wget -O eggnog.db.gz \
  http://eggnog5.embl.de/download/emapperdb-5.0.2/\
eggnog.db.gz
gunzip eggnog.db.gz

# Diamond protein database (4.8 GB compressed, ~8.7 GB)
wget -O eggnog_proteins.dmnd.gz \
  http://eggnog5.embl.de/download/emapperdb-5.0.2/\
eggnog_proteins.dmnd.gz
gunzip eggnog_proteins.dmnd.gz

# Taxonomy database (~71 MB compressed)
wget -O eggnog.taxa.tar.gz \
  http://eggnog5.embl.de/download/emapperdb-5.0.2/\
eggnog.taxa.tar.gz
tar xzf eggnog.taxa.tar.gz
```

Total disk usage: approximately 48 GB.

3. Verify installation.

```
docker run --rm \
  -v /path/to/eggnog-data:/data \
  quay.io/biocontainers/\
    eggnog-mapper:2.1.13--pyhdfd78af_2 \
```

```
  emapper.py --version --data_dir /data

# Output:
# emapper-2.1.13 / eggNOG DB version: 5.0.2
# Diamond version: 2.0.15 / MMseqs2: 16.747c6
```

B.3 Running eggNOG-mapper

```
docker run --rm \
  -v /path/to/eggnog-data:/data \
  -v /path/to/input:/input \
  -v /path/to/output:/output \
  quay.io/biocontainers/\
    eggnog-mapper:2.1.13--pyhdfd78af_2 \
  emapper.py \
    -i /input/test_proteins.fasta \
    --data_dir /data \
    -o test_proteins \
    --output_dir /output \
    --cpu 8 \
    -m diamond \
    --go_evidence experimental \
    --tax_scope auto \
    --target_orthologs all
```

Key parameters.

`-m diamond` Use DIAMOND for sequence search (default mode).

`-go_evidence experimental` Transfer only GO terms supported by experimental evidence codes, matching the evidence code filter used in PROTEA's evaluation protocol.

`-tax_scope auto` Automatically determine the taxonomic scope for ortholog assignment.

`-target_orthologs all` Consider all orthologs (one-to-one, one-to-many, many-to-many).

`-cpu 8` Use 8 CPU threads for the DIAMOND search phase.

Runtime. The full run completed in approximately 21 minutes on the same workstation used for all PROTEA experiments (64 GB RAM, no GPU required for eggNOG-mapper).

Output. eggNOG-mapper produced annotations for 19,958 of the 20,281 input proteins (98.4% hit rate). Of those, 17,334 (85.5% of the test set) received at least one GO term prediction. The remaining 2,947 proteins either had no DIAMOND hit or no GO terms were transferred from the matched orthologous group.

B.4 CAFA Evaluation Protocol

To ensure a fair comparison, eggNOG-mapper predictions were evaluated using exactly the same protocol as all PROTEA experiments:

1. **Ground truth computation.** The NK/LK/PK classification and ground-truth annotations were computed from the same EvaluationSet (GOA 220→229) using PROTEA's `compute_evaluation_data()` function, which implements NOT-qualifier propagation through the GO DAG following the CAFA5 protocol.

2. **Prediction format.** eggNOG-mapper does not produce per-term confidence scores; each predicted GO term is assigned a uniform score of 1.0 (binary: predicted or not predicted).

3. **Evaluation.** The `cafaeval` tool was run with the same parameters as all other experiments: GO propagation mode `max`, normalisation mode `cafa`, and IA weighting from `IA_cafa6.tsv`.

The evaluation script `scripts/evaluate_external_tool.py` automates this entire pipeline. It accepts the external tool's output file, connects to PROTEA's database to retrieve the ground truth, and runs `cafaeval` for all three CAFA settings (NK, LK, PK):

```
poetry run python scripts/evaluate_external_tool.py \
  --evaluation-set-id \
    42b34e79-6fe9-4fa0-b718-02f43a1e3192 \
  --tool emapper \
  --input output/test_proteins.emapper.annotations
```

B.5 Raw Results

Table B.1 presents the complete per-cell results for eggNOG-mapper alongside the PROTEA methods.

Table B.1: Complete $F_{\max}$ comparison: eggNOG-mapper vs. PROTEA methods (IA-weighted).

	NK			**LK**			**PK**		
Method	BPO	MFO	CCO	BPO	MFO	CCO	BPO	MFO	CCO
eggNOG-mapper 2.1.13	0.247	0.359	0.386	0.382	0.334	0.450	0.190	0.199	0.325
PROTEA baseline	0.412	0.590	0.668	0.467	0.558	0.675	0.187	0.278	0.325
PROTEA re-ranker v3	**0.431**	**0.620**	**0.692**	**0.478**	**0.607**	**0.697**	**0.201**	**0.297**	**0.339**

Table B.2 shows the absolute $F_{\max}$ difference between PROTEA's re-ranker v3 and eggNOG-mapper.

Table B.2: Absolute $F_{\max}$ improvement: PROTEA re-ranker v3 vs. eggNOG-mapper.

Category	**BPO**	**MFO**	**CCO**
NK	+0.184	+0.261	+0.306
LK	+0.096	+0.273	+0.247
PK	+0.011	+0.098	+0.014

Coverage analysis. eggNOG-mapper found DIAMOND hits for 19,958 proteins but only transferred GO terms to 17,334 (85.5% of the test set). PROTEA, by contrast, produces predictions for all 20,281 proteins (100% coverage) because every protein has an embedding and therefore has k nearest neighbours. This coverage gap accounts for part of the $F_{\max}$ difference, particularly in NK where 14.5% of proteins receive no eggNOG-mapper prediction.

B.6 Methodological Notes

Score assignment. eggNOG-mapper produces a binary set of GO terms per protein without per-term confidence scores. For the CAFA evaluation, all predicted terms are assigned a confidence of 1.0. This means the precision-recall curve is a single point (one threshold), and $F_{\max}$ equals the F_1 score at that point. Methods that produce graded confidence scores (such as PROTEA) can optimise the precision-recall trade-off by varying the threshold, which may contribute to higher $F_{\max}$ values.

GO propagation. eggNOG-mapper transfers GO terms from orthologous groups but does not explicitly propagate them up the GO DAG. The `cafaeval` evaluator applies `prop=max` propagation during evaluation, so ancestor terms are included in the comparison for all methods.

Evidence code filter. The `-go_evidence experimental` flag restricts eggNOG-mapper to transferring GO terms supported by experimental evidence codes in the eggNOG database. This is conceptually aligned with PROTEA's use of experimental-only annotations from GOA, though the exact set of supporting evidence and the database versions differ.